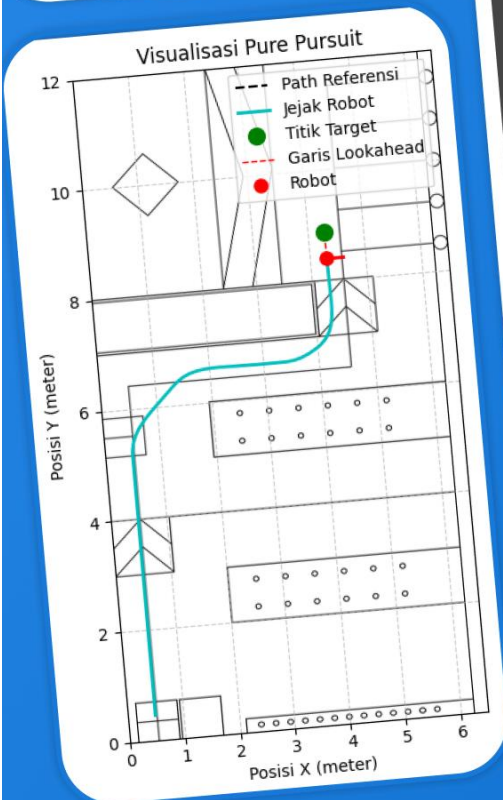
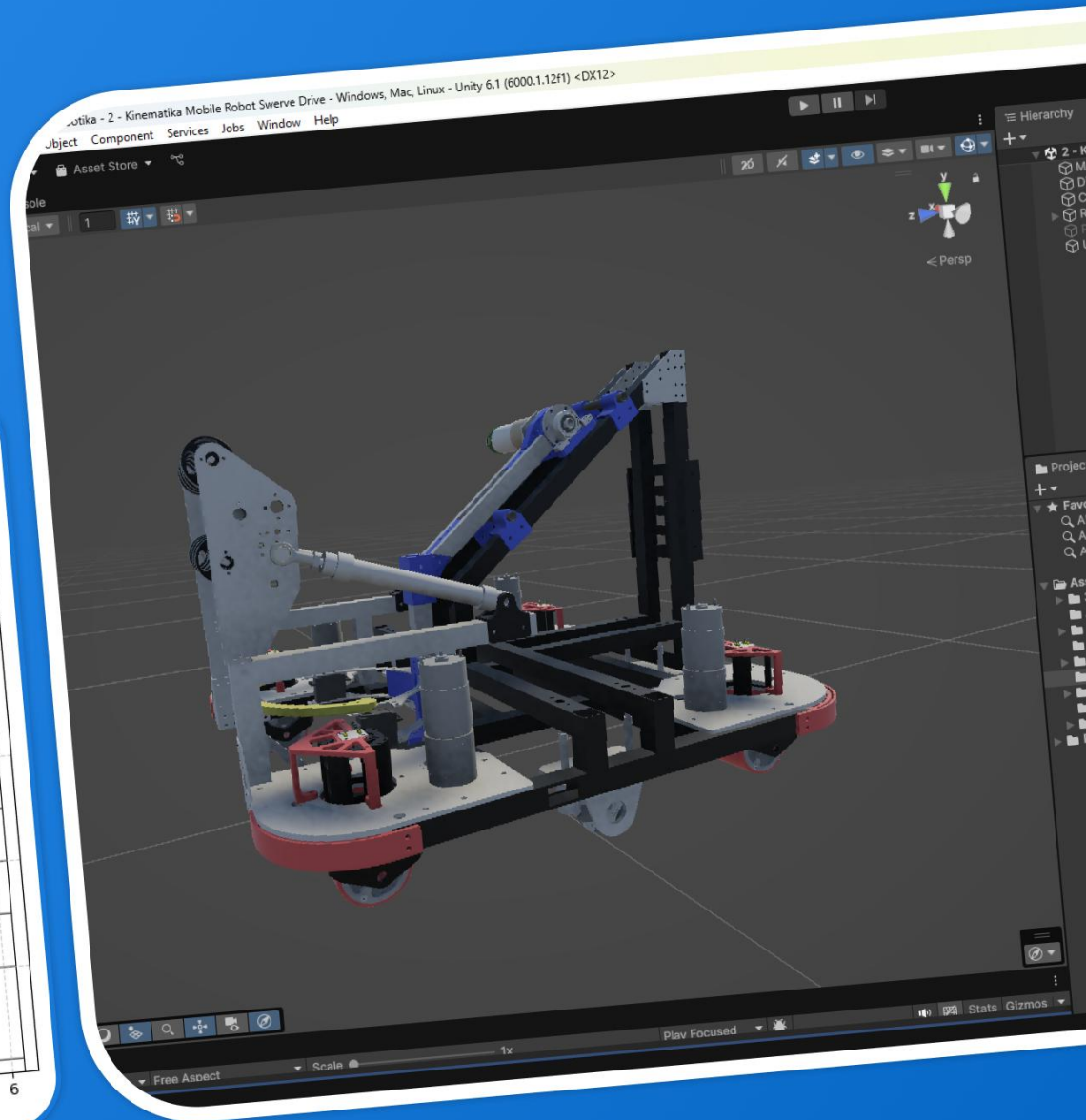
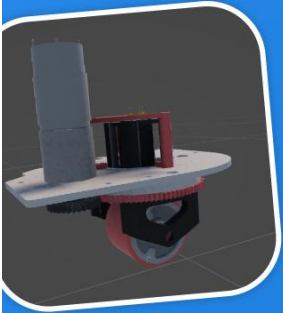




DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
UNIVERSITAS NEGERI YOGYAKARTA



MODUL PRAKTIK ROBOTIKA

Simulator Kinematika & Navigasi Swerve Drive



Vincent Kenutama Prasetyo
21518241007

Dr. Herlambang Sigit Pramono, ST, M.Cs
196508291999031001

KATA PENGANTAR

Dengan mengucapkan syukur kepada Tuhan Yesus Kristus yang telah melimpahkan segala anugerah dan karunia-Nya sehingga penulis dapat menyelesaikan penyusunan modul pembelajaran dengan judul “Pengembangan Simulator Kinematika dan Navigasi *Swerve Drive Autonomous Mobile Robot* Berbasis Unity 3D” sebagai salah satu pelengkap media pembelajaran robotika.

Modul pembelajaran ini dapat terselesaikan dengan baik tidak lepas dari dukungan, bimbingan, serta saran dari berbagai pihak, di antaranya Bapak Herlambang Sigit. P, S.T. M.Cs yang telah membimbing Penulis dalam penyusunan modul pembelajaran ini. Kepada pihak-pihak yang telah berkontribusi dalam penyusunan modul ini yang tidak dapat disebutkan satu-persatu penulis mengucapkan terima kasih atas segala bantuan yang telah diberikan.

Penulis sadar akan kekurangan pada modul ini, oleh karena hal tersebut kritik dan saran dari pembaca sangat berarti untuk pengembangan selanjutnya. Akhir kata, penulis harap modul ini dapat bermanfaat bagi kemajuan teknologi khususnya di bidang robotika dan dapat dijadikan sebagai referensi untuk pengembangan sejenis.

Yogyakarta, 14 Juli 2025

Vincent Kenutama Prasetyo
NIM. 21518241007

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI	2
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
BAB I PENDAHULUAN.....	6
A. Mata Kuliah Praktik Robotika	6
B. Desain Pengembangan Media.....	8
BAB II MATERI.....	13
A. Dasar Teori	13
1. Swerve Drive.....	13
2. Pengantar Kinematika Robot	14
3. Kinematika Mundur (<i>Inverse Kinematics</i>).....	15
4. Kinematika Maju (<i>Forward Kinematics</i>).....	15
5. Model Kinematika <i>Swerve Drive</i>	16
6. Lokalisasi Odometry Tiga Roda.....	24
7. PID Controllers	28
8. Pure Pursuit	31
B. Komponen Perangkat Lunak (<i>Software</i>)	33
1. Simulasi Kinematika Robot Berbasis Unity	33
2. Visual Studio Code.....	36
C. Instalasi Visual Studio Code	38
D. Langkah Persiapan Pembelajaran	42
E. Navigasi pada <i>Unity Editor Scene View</i>	59
F. Pemrograman GUI dan <i>Data Acquisition</i> Python.....	64
G. Kode Program Python	67
1. Kode Python Pembelajaran PID Module Swerve Drive	67
2. Kode Python Pembelajaran Kinematika dan Odometry	75
3. Kode Python Pembelajaran Navigasi Autonomous Pure Pursuit.....	80
H. Implementasi Model Kinematika pada Simulasi	86
1. Implementasi Kinematika Mundur	86

2. Implementasi Kinematika Maju	88
DAFTAR PUSTAKA	91

DAFTAR GAMBAR

Gambar 1. Grafik Respons Sistem (a) Visualisasi Pengendalian Sudut Modul <i>Swerve Drive</i> pada Simulasi (b).....	9
Gambar 2. Plot Jejak Odometri (a) Visualisasi Pergerakan Robot pada Simulasi (b)	11
Gambar 3. Simulasi Navigasi Otonom <i>Pure Pursuit</i> (a) Visualisasi Jejak Robot (b)	12
Gambar 4. Modul <i>Swerve Drive</i>	14
Gambar 5. Kinematika <i>Swerve Drive</i> 4 Roda.....	17
Gambar 6. Konfigurasi Odometry Tiga Roda	24
Gambar 7. Diagram Blok Kendali Proportional	29
Gambar 8. Diagram Blok Kendali Proportional-Integral.....	30
Gambar 9. Diagram Blok Kendali Proportional-Derivative.....	31
Gambar 10. Algoritma <i>Pure Pursuit</i>	32
Gambar 11. Pengaruh Parameter Look Ahead Distance (a) kecil (b) besar.....	33
Gambar 12. Software Unity 3D	33
Gambar 13 Antarmuka Visual Studio Code	37

DAFTAR TABEL

Tabel 1. Fungsi Tombol View Tools di Unity	61
--	----

BAB I

PENDAHULUAN

A. Mata Kuliah Praktik Robotika

Mata kuliah Praktik Robotika merupakan salah satu pilar fundamental dalam kurikulum program studi Pendidikan Teknik Mekatronika. Mata kuliah ini berfungsi sebagai jembatan krusial yang menghubungkan ranah teoris dengan aplikasi dunia nyata, menuntut mahasiswa untuk mengintegrasikan pengetahuan dari berbagai disiplin ilmu seperti rekayasa mekanik, teknik elektronika, dan ilmu komputer ke dalam sebuah proyek rekayasa yang terpadu. Tujuan utama dari mata kuliah ini adalah untuk membekali mahasiswa dengan pengalaman langsung dalam siklus pengembangan robot, mulai dari konseptualisasi, perancangan mekanik, perakitan sirkuit elektronika, hingga algoritma perangkat lunak yang kompleks.

Setiap robot dikembangkan berdasarkan tiga pilar utama yaitu: mekanik, elektronika, dan program. Pilar mekanik mencakup struktur fisik seperti rangka (*body*), sasis (*base*), dan sistem penggerak. Pilar elektronika berfungsi sebagai sistem saraf, meliputi komponen seperti mikrokontroler, sensor, dan sirkuit daya. Terakhir, pilar program adalah otak dari robot, berisi logika dan algoritma yang mengendalikan perilaku robot dalam menyelesaikan tugas yang diberikan.

Dalam dunia robotika terdapat banyak jenis penggerak yang dipelajari, salah satu contohnya adalah sistem penggerak berbasis *Swerve Drive* yang menjadi salah satu studi kasus yang menantang dan relevan dengan industri robotika modern. Teknologi ini menawarkan fleksibilitas gerak holonomik yaitu robot dapat bergerak ke segala arah sambil berotasi secara independen. Kemampuan ini dapat dicapai dengan memberikan setiap modul roda dua derajat kebebasan: kemampuan untuk mengatur sudut (*steering*) dan kemampuan untuk berputar menggerakkan robot. Keunggulan manuver ini menjadikan *Swerve Drive* solusi ideal untuk aplikasi yang menuntut pergerakan cepat dan presisi di lingkungan yang kompleks seperti pada permainan ABU Robocon 2024.

Keunggulan manuver *Swerve Drive* ini diimbangi dengan kompleksitas yang tinggi pada level kontrol dan pemrogramannya. Untuk dapat mengendalikannya, peserta didik harus memiliki pengetahuan mengenai model kinematika untuk menerjemahkan perintah gerak robot secara keseluruhan (maju, mundur, ke samping, dan berputar) yang menjadi perintah sudut dan kecepatan individual pada setiap motornya. Setiap modul roda memerlukan kontrol *loop* tertutup yang presisi.

Menghadapi kompleksitas ini secara langsung pada perangkat keras fisik dalam kerangka waktu Praktik Robotika dapat menjadi tidak efisien dan menimbulkan beberapa kendala belajar. Proses *debugging* menjadi sangat sulit ketika harus membedakan antara kesalahan pada model matematika, implementasi kode, atau malfungsi perangkat keras. Oleh

karena itu, pengembangan media pembelajaran berbasis simulator menjadi solusi untuk menjembatani kesenjangan ini.

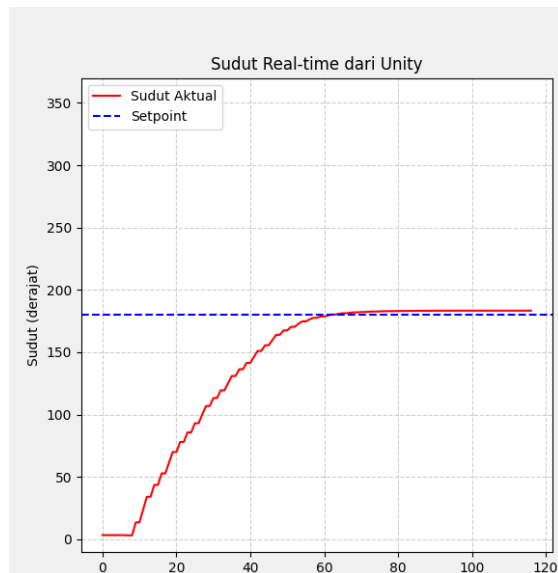
B. Desain Pengembangan Media

Pengembangan simulator kinematika dan navigasi *Swerve Drive Autonomous Mobile Robot* menggunakan perangkat lunak *game engine* Unity 3D sebagai *platform* utama untuk simulasi fisika dan visualisasi 3D menggunakan bahasa pemrograman C#. Pengambilan data dilakukan dengan bahasa pemrograman Python dengan *library* Matplotlib dan Tkinter untuk membangun *dashboard* kontrol dan analisis data eksternal pada simulasi. Simulator memberikan data telemetri secara *real-time* kepada program Python sehingga dapat dilakukan uji coba dan pengamatan terhadap parameter yang diberikan pada robot seperti PID untuk pengendalian sudut modul Swerve Drive, Odometry pergerakan robot, dan *Look ahead gain* pada *Pure Pursuit*.

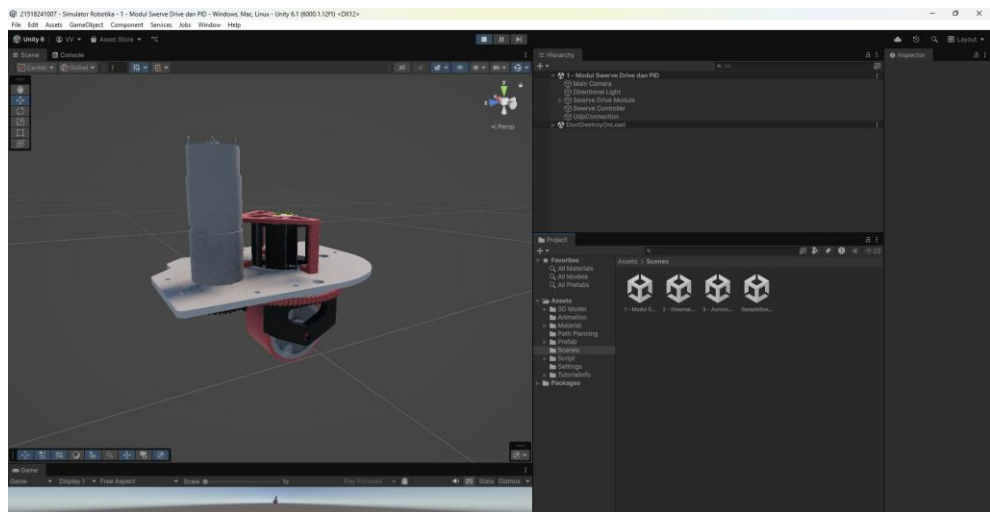
Dalam pembelajaran terbagi menjadi tiga bagian utama yaitu: Pengendalian Sudut Modul *Swerve Drive*, Implementasi Kinematika *Swerve Drive* dan Odometry, serta Navigasi *Autonomous Mobile Robot* dengan *Pure Pursuit* menggunakan jalur pada perlombaan ABU Robocon 2024.

Pengendalian sudut modul Swerve Drive berfokus pada unit terkecil dari sistem penggerak *Swerve Drive*, yaitu satu modul roda untuk memahami dan menguasai implementasi sistem kontrol *loop* tertutup untuk

motor *steering*. Gambar 1 merupakan contoh penggunaan media pembelajaran untuk pengendalian sudut modul *Swerve Drive*.



(a)

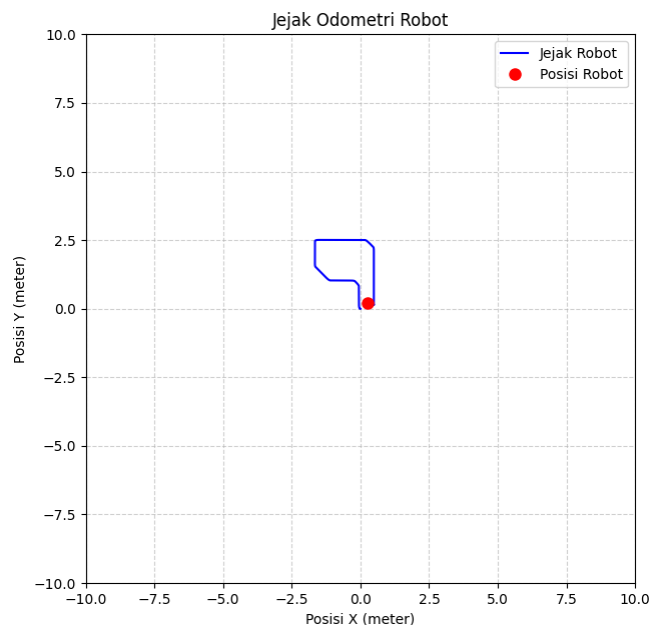


(b)

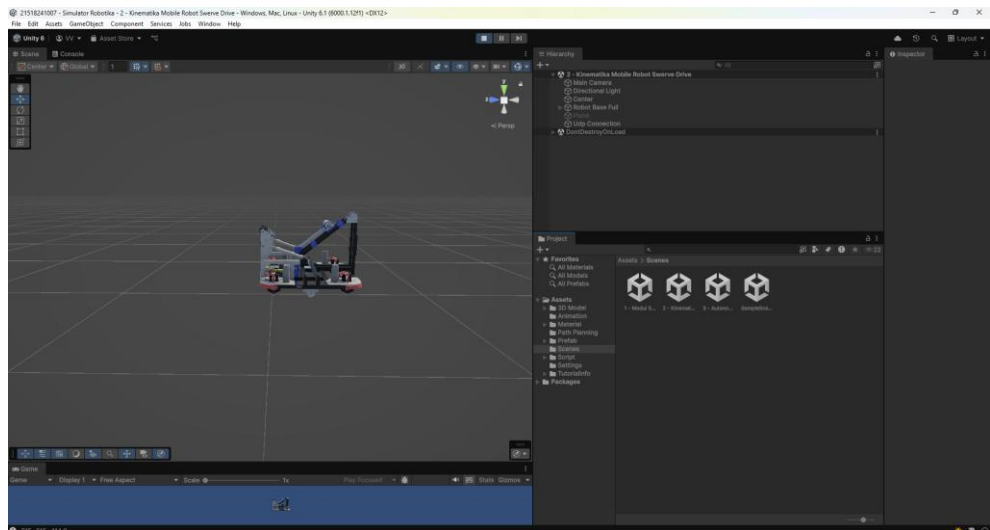
Gambar 1. Grafik Respons Sistem (a) Visualisasi Pengendalian Sudut Modul *Swerve Drive* pada Simulasi (b)

Pada Gambar 1a menunjukkan pengaruh parameter PID yang diberikan pada modul, sedangkan pada Gambar 1b memberikan visualisasi pergerakan modul roda *Swerve Drive* yang dapat diamati secara 3 dimensi.

Setelah peserta didik menguasai kontrol modul secara individu, langkah berikutnya adalah mengintegrasikan keempat modul roda dan mengimplementasikan model kinematika yang memungkinkan robot bergerak sebagai kesatuan yang terkoordinasi. Dalam *script* C# di Unity, peserta didik menulis model *inverse kinematics* yang menerima tiga perintah gerak global: kecepatan maju/mundur (Vx), kecepatan gerak menyamping/*strafe* (Vy), dan kecepatan rotasi (ω). Setelah kinematika balik dibuat, peserta didik juga membuat kinematika untuk *Odometry* menggunakan sistem tiga roda, setelah semua selesai hasilnya dapat divalidasi menggunakan program Python sebagai data *logger* dari telemetri Unity yang ditunjukkan pada Gambar 2b, pergerakan robot juga dapat diamati pada Gambar 2a.



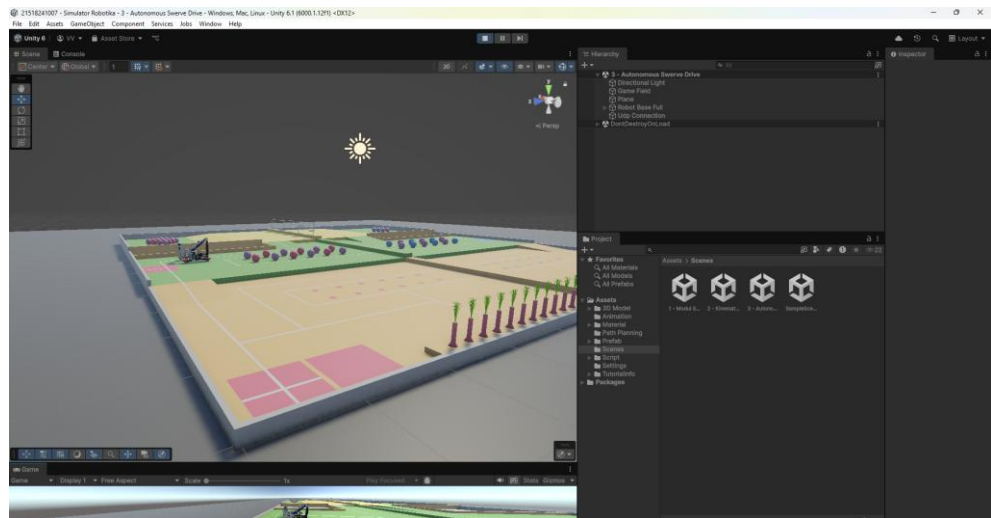
(a)



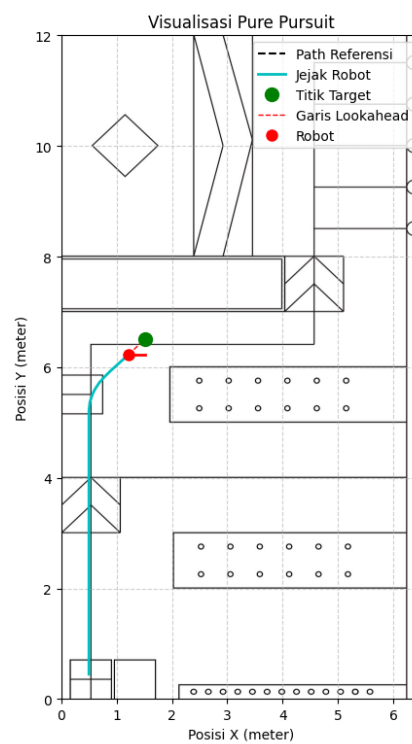
(b)

Gambar 2. Plot Jejak Odometri (a) Visualisasi Pergerakan Robot pada Simulasi (b)

Tahap terakhir dari modul pembelajaran ini adalah navigasi otonom dengan *Pure Pursuit*. Setelah robot dapat dikendalikan secara presisi melalui model kinematika dan posisinya dapat dilacak melalui odometri, mahasiswa ditantang untuk membuat robot bergerak secara otomatis mengikuti lintasan yang mereferensi pada perlombaan ABU Robocon 2024. Mahasiswa diminta mengimplementasikan algoritma *Pure Pursuit* secara kontinu untuk menghitung sebuah titik target (*lookahead point*) dengan mengubah parameter *look ahead* dan *gain*. Lintasan yang dilalui oleh robot akan terekam dalam program Python melalui telemetri yang dilakukan, sehingga pengaruh kedua parameter tersebut dapat diamati kembali melalui grafik seperti pada Gambar 3b.



(a)



Gambar 3. Simulasi Navigasi Otonom *Pure Pursuit* (a) Visualisasi Jejak Robot (b)

BAB II

MATERI

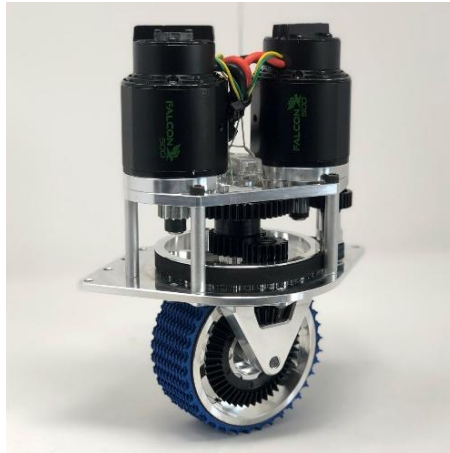
A. Dasar Teori

1. Swerve Drive

Swerve Drive dikenal juga sebagai *Crab Drive* atau *Coaxial Drive* adalah sistem penggerak holonomik yang memberikan mobilitas pada *Mobile Robot*. Robot disebut sebagai holonomik apabila jumlah derajat kebebasan yang dapat dikendalikan sama dengan total derajat kebebasan di ruang kerjanya. Robot di bidang 2 dimensi (2D) berarti robot dapat bergerak secara instan ke arah manapun termasuk maju, mundur, ke samping kiri, maupun kanan dan berotasi secara bersamaan, tanpa perlu melakukan manuver berbelok. Terdapat dua prinsip utama Swerve Drive adalah penggunaan modul-modul roda independen, tiap modul terdiri dari dua aktuator berupa motor:

- a. Motor penggerak (*Drive Motor*) untuk mengontrol kecepatan putaran roda untuk menggerakkan robot.
- b. Motor belok (*Steering Motor*) untuk mengendalikan orientasi (sudut) roda terhadap sasis robot.

Dengan kemampuan untuk mengarahkan setiap roda secara independen, robot dapat mencapai pergerakan *omnidirectional* yang sesungguhnya.



Gambar 4. Modul Swerve Drive
(Sumber: <https://www.swervedrivespecialties.com/>)

2. Pengantar Kinematika Robot

Kinematika robot adalah studi yang mempelajari gerak robot berfokus pada hubungan antara konfigurasi sudut-sudut sendi robot dan posisi serta orientasi pada robot tanpa memperhitungkan gaya penyebab gerakan tersebut (Villaverde & Maneetham, 2024). Dalam robotika, pemodelan kinematika adalah langkah fundamental yang memungkinkan perangkat lunak berinteraksi dengan dunia fisik seperti gerakan motor dan roda.

Terdapat dua jenis model kinematika yang bekerja dan saling melengkapi, yaitu Kinematika Mundur (*Inverse Kinematics*) yang digunakan untuk mengontrol robot, sementara Kinematika Maju (*Forward Kinematics*), yang diimplementasikan sebagai Odometri, digunakan untuk melacak posisi robot.

3. Kinematika Mundur (*Inverse Kinematics*)

Pada *mobile robot*, kinematika mundur merupakan proses dalam menentukan kecepatan putar masing-masing roda agar robot dapat bergerak sesuai arah dan posisi yang diinginkan, termasuk kemampuan dalam bergerak ke segala arah tanpa harus berbelok seperti pada roda konvensional (Fahmizal et al., 2022). Dalam kinematika mundur, persamaan matematika *inverse kinematics* digunakan untuk mengubah kecepatan linear dan sudut putar robot menjadi kecepatan dan sudut pada roda.

Kinematika mundur adalah proses menerjemahkan satu set perintah gerak global menjadi beberapa set perintah individual untuk setiap aktuator (dalam hal ini, motor *steering* dan *drive* pada setiap modul roda). Model ini menjadi inti dari logika gerak robot. *Input* seperti gerak maju harus diproses melalui model kinematika mundur sebelum dapat dieksekusi oleh robot. Proses perhitungan kinematika mundur melibatkan dekomposisi vektor kecepatan yang diinginkan menjadi komponen-komponen yang dirasakan oleh setiap modul roda. Vektor kecepatan total untuk setiap roda (V_i) adalah hasil penjumlahan dari vektor translasi (V_{trans}) dan vektor rotasi (V_{rot}).

4. Kinematika Maju (*Forward Kinematics*)

Kinematika maju pada *mobile robot* merupakan proses perhitungan posisi dan orientasi robot berdasarkan nilai kecepatan linier dan sudut

rotasi roda sebagai *input* kendali (Ilhan Shanjaya et al., 2024). Pada robot beroda, kinematika maju umumnya digunakan untuk menentukan perpindahan posisi dan arah gerak robot dalam koordinat global. Model kinematika maju mengonversi kecepatan sudut masing-masing roda ke perubahan posisi (x,y) dan orientasi (θ) robot, sehingga dapat memprediksi lintasan dan gerakan robot.

Kinematika ini merupakan proses kebalikan dari kinematika mundur karena menggunakan keadaan aktuator individual untuk menentukan keadaan global dari seluruh robot. Dalam konteks *mobile robot*, aplikasi praktis dari kinematika maju adalah Odometri, yaitu proses estimasi posisi dan orientasi robot secara terus menerus berdasarkan data dari sensor. Model ini digunakan untuk melacak atau melokalisasi posisi robot, memberikan umpan balik krusial yang dibutuhkan oleh algoritma navigasi tingkat tinggi.

5. Model Kinematika *Swerve Drive*

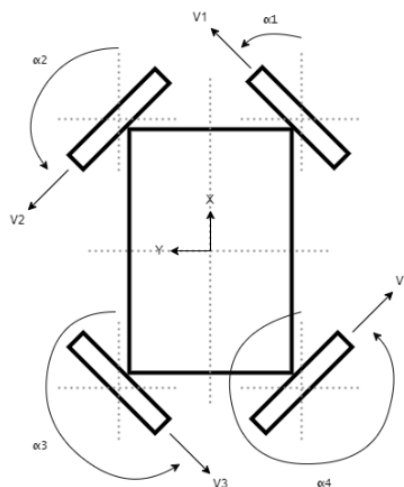
Dalam konteks *Swerve Drive*, kinematika menghubungkan kecepatan dan sudut setiap roda dengan kecepatan gerak robot secara keseluruhan. Terdapat dua jenis kinematika antara lain:

- a. Kinematika Balik (*Inverse Kinematics*) adalah kinematika yang menentukan kecepatan dan sudut setiap roda individu (v_i , θ_i) berdasarkan *input* gerak robot yang diinginkan (v_x , v_y , ω).

Perhitungan kinematika ini digunakan untuk mengendalikan robot dan menjadi inti dari logika gerak robot.

- b. Kinematika Maju (*Forward Kinematics*) adalah kinematika yang menentukan gerak robot secara keseluruhan (v_x, v_y, ω) berdasarkan data kecepatan dan sudut dari setiap roda individu (v_i, θ_i).

Swerve Drive memungkinkan setiap roda untuk berotasi terhadap sumbu vertikalnya untuk berbelok dan secara bersamaan berputar pada sumbu horizontalnya untuk bergerak. Kombinasi dua gerakan ini menghasilkan mobilitas *omnidirectional* untuk mengendalikan gerakan secara presisi dengan pemodelan matematika.



Gambar 5. Kinematika Swerve Drive 4 Roda

(Sumber: <https://www.petrikvandervelde.nl/posts/Swerve-drive-kinematics-simulation>)

Sebelum menurunkan persamaan kinematika, penting untuk mendefinisikan kerangka acuan dan notasi yang akan digunakan, modul

swerve drive disusun dalam konfigurasi persegi panjang seperti ilustrasi pada Gambar 5. Notasi yang digunakan antara lain:

- a. Kerangka acuan robot (*Body Frame*), merupakan titik pada koordinat (0,0) berada di tengah geometris dari sasis.
- b. Sumbu koordinat menunjukkan arah gerak robot, sumbu +X menunjukkan ke arah kanan robot, sedangkan sumbu +Y menunjukkan arah ke depan robot. Rotasi diukur mengelilingi sumbu Z yang menunjuk ke atas.
- c. Terdapat dua parameter sasis antara lain L yang merupakan jarak total antara roda depan dan belakang (panjang) dan W yang mendefinisikan jarak total antara roda kiri dan kanan (lebar).
- d. Vektor kecepatan robot adalah kecepatan gerak robot yang diwakili dengan tiga komponen antara lain v_x yang merupakan kecepatan translasi ke arah samping sepanjang sumbu X, v_y yang mewakili kecepatan translasi ke arah depan sepanjang sumbu Y, dan ω merupakan kecepatan rotasi (angular) mengelilingi pusat robot.

Setiap modul roda dapat diidentifikasi berdasarkan posisinya Depan Kanan (FR), Depan Kiri (FL), Belakang Kiri (RL), dan Belakang Kanan (RR). *Inverse kinematics* bertujuan untuk menghitung kecepatan (v_i) dan sudut (θ_i) yang harus dicapai oleh setiap modul roda untuk menghasilkan gerakan robot yang diinginkan. Gerakan pada setiap titik benda tegar merupakan gabungan dari gerak translasi dan rotasi. Oleh karena itu, vektor kecepatan pada lokasi setiap modul roda (v_i) adalah

jumlah vektor kecepatan translasi seluruh robot dan vektor kecepatan tangensial yang disebabkan oleh rotasi robot, dapat dirumuskan menjadi

$$v_i = v_{translasi} + v_{rotasi}$$

Keterangan:

v_i = Vektor kecepatan tiap roda

Vektor kecepatan rotasi dihitung sebagai produk silang antara kecepatan angular dan vektor posisi (jarak dari pusat robot ke roda). Dalam sistem 2 dimensi, hal ini dapat disederhanakan lebih lanjut menjadi:

$$v_{ix} = v_x - \omega * r_{iy}$$

$$v_{iy} = v_y + \omega * r_{ix}$$

Keterangan:

v_{ix} = Kecepatan roda ke-i pada sumbu X

v_{iy} = Kecepatan roda ke-i pada sumbu Y

v_x = Kecepatan translasi pusat robot pada sumbu X

v_y = Kecepatan translasi pusat robot pada sumbu Y

r_{ix} = Jarak roda ke-i terhadap pusat robot pada sumbu X

r_{iy} = Jarak roda ke-i terhadap pusat robot pada sumbu Y

Dalam hal ini, r_{ix} dan r_{iy} adalah koordinat posisi X dan Y dari roda ke-i relatif terhadap pusat robot. Berdasarkan parameter sasis L dan W, posisi setiap roda adalah:

$$\text{Depan Kanan (FR)} = \left(\frac{L}{2}, \frac{W}{2}\right)$$

$$\text{Depan Kiri (FL)} = \left(-\frac{L}{2}, \frac{W}{2}\right)$$

$$\text{Belakang Kiri (RL)} = \left(-\frac{L}{2}, -\frac{W}{2}\right)$$

$$\text{Belakang Kanan (RR)} = \left(\frac{L}{2}, -\frac{W}{2}\right)$$

Keterangan:

L = Panjang sasis robot (jarak antara roda depan dan belakang)

W = Lebar sasis robot (jarak antara roda kanan dan kiri)

Dengan mensubstitusikan koordinat-koordinat ini ke dalam persamaan umum, dapat ditentukan vektor kecepatan untuk setiap roda, antara lain:

a. Persamaan roda depan kanan (FR)

$$v_{FRx} = v_x - \omega * \left(\frac{W}{2}\right)$$

$$v_{FRy} = v_y + \omega * \left(\frac{L}{2}\right)$$

$$v_{FR} = \sqrt{\left(v_x - \omega * \left(\frac{W}{2}\right)\right)^2 + \left(v_y + \omega * \left(\frac{L}{2}\right)\right)^2}$$

$$\theta_{FR} = \arctan2 \left(\frac{v_y + \omega \cdot \frac{L}{2}}{v_x - \omega \cdot \frac{W}{2}} \right)$$

Keterangan:

v_{FRx} = Kecepatan roda depan kanan pada sumbu x

v_{FRy} = Kecepatan roda depan kanan pada sumbu y

v_{FR} = Resultan kecepatan roda kanan, gabungan dari v_{FRx} dan v_{FRy}

θ_{FR} = Sudut kemudi roda depan kanan terhadap sumbu x

b. Persamaan roda depan kiri (FL)

$$v_{FLx} = v_x - \omega * \left(\frac{W}{2} \right)$$

$$v_{FLy} = v_y - \omega * \left(\frac{L}{2} \right)$$

$$v_{FL} = \sqrt{\left(v_x - \omega * \left(\frac{W}{2} \right) \right)^2 + \left(v_y - \omega * \left(\frac{L}{2} \right) \right)^2}$$

$$\theta_{FL} = \arctan2 \left(\frac{v_y - \omega \cdot \frac{L}{2}}{v_x - \omega \cdot \frac{W}{2}} \right)$$

Keterangan:

v_{FLx} = Kecepatan roda depan kiri pada sumbu x

v_{FLy} = Kecepatan roda depan kiri pada sumbu y

v_{FL} = Resultan kecepatan roda kiri, gabungan dari v_{FLx} dan v_{FLy}

θ_{FL} = Sudut kemudi roda depan kiri terhadap sumbu x

c. Persamaan roda belakang kiri (RL)

$$v_{RLx} = v_x + \omega * \left(\frac{W}{2}\right)$$

$$v_{RLy} = v_y - \omega * \left(\frac{L}{2}\right)$$

$$v_{RL} = \sqrt{\left(v_x + \omega * \left(\frac{W}{2}\right)\right)^2 + \left(v_y - \omega * \left(\frac{L}{2}\right)\right)^2}$$

$$\theta_{RL} = \arctan2 \left(\frac{v_y - \omega * \frac{L}{2}}{v_x + \omega * \frac{W}{2}} \right)$$

Keterangan:

v_{RLx} = Kecepatan roda belakang kiri pada sumbu x

v_{RLy} = Kecepatan roda belakang kiri pada sumbu y

v_{RL} = Resultan kecepatan roda belakang kiri, gabungan dari v_{RLx}

dan v_{RLy}

θ_{RL} = Sudut kemudi roda belakang kiri terhadap sumbu x

d. Persamaan roda belakang kanan (RR)

$$v_{RRx} = v_x + \omega * \left(\frac{W}{2}\right)$$

$$v_{RRy} = v_y + \omega * \left(\frac{L}{2}\right)$$

$$v_{RR} = \sqrt{\left(v_x + \omega * \left(\frac{W}{2}\right)\right)^2 + \left(v_y + \omega * \left(\frac{L}{2}\right)\right)^2}$$

$$\theta_{RR} = \arctan2 \left(\frac{v_y + \omega \cdot \frac{L}{2}}{v_x + \omega \cdot \frac{W}{2}} \right)$$

Keterangan:

v_{RLx} = Kecepatan roda belakang kiri pada sumbu x

v_{RLy} = Kecepatan roda belakang kiri pada sumbu y

v_{RL} = Resultan kecepatan roda belakang kiri, gabungan dari v_{RLx} dan v_{RLy}

θ_{RL} = Sudut kemudi roda belakang kiri terhadap sumbu x

Dengan demikian, kecepatan dan sudut setiap roda dapat ditulis secara lebih ringkas:

a. Roda depan kanan (FR)

$$V_{FR} = \sqrt{(B^2 + D^2)}$$

$$\theta_{FR} = \text{atan2} \left(\frac{D}{B} \right)$$

b. Roda depan kiri (FL)

$$V_{FL} = \sqrt{(A^2 + D^2)}$$

$$\theta_{FL} = \text{atan2} \left(\frac{D}{A} \right)$$

c. Roda belakang kiri (RL)

$$V_{RL} = \sqrt{(A^2 + C^2)}$$

$$\theta_{RL} = \text{atan2} \left(\frac{C}{A} \right)$$

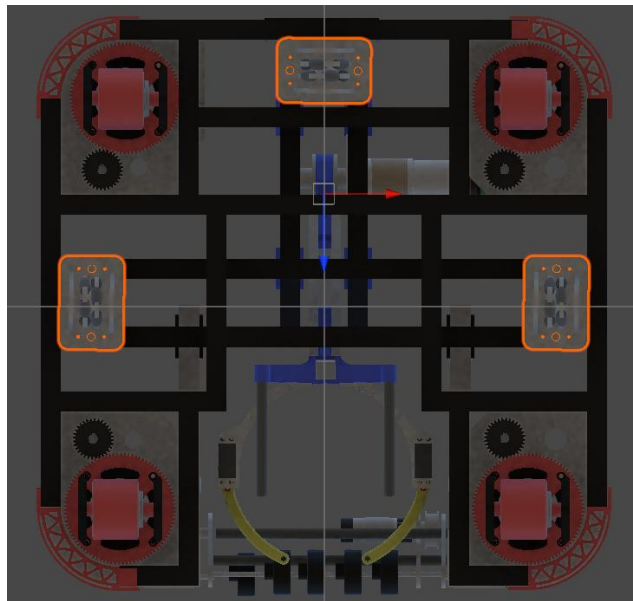
d. Roda belakang kanan (RR)

$$V_{RR} = \sqrt{(B^2 + C^2)}$$

$$\theta_{RR} = \text{atan2} \left(\frac{C}{B} \right)$$

Pemodelan yang disederhanakan di atas memungkinkan secara matematis dan lebih efisien untuk dihitung dalam perangkat lunak simulator.

6. Lokalisasi Odometry Tiga Roda



Gambar 6. Konfigurasi Odometry Tiga Roda

Odometry merupakan metode yang digunakan dalam memperkirakan posisi dan orientasi robot pada lapangan berdasarkan sensor seperti *wheel encoder*. Teknik ini digunakan untuk navigasi robotika, terutama untuk *mobile robot* yang bergerak di lingkungan yang diketahui maupun belum diketahui sebelumnya (Fachrizi et al., 2024).

Odometry digunakan untuk memperkirakan posisi relatif terhadap posisi awal, digunakan sensor *rotary encoder* yang menghasilkan perhitungan jumlah pulsa yang dihasilkan kemudian dikonversi menjadi satuan milimeter. Berikut merupakan persamaan yang digunakan untuk menentukan:

$$\text{Keliling roda } (K) = 2 * \pi * r$$

$$\text{Pulsa per mm} = \frac{\text{Resolusi Encoder}}{K}$$

Robot yang menggunakan roda *omnidirectional* dapat digunakan persamaan untuk mendapatkan koordinat X dan Y. Pada sistem *omni* tiga roda, perhitungan posisi menggunakan model kinematika *omnidirectional*, di mana kecepatan tiap roda diolah untuk mendapatkan perubahan posisi ($\Delta x, \Delta y$) dan perubahan orientasi ($\Delta \theta$). Akurasi *odometry* dapat ditingkatkan dengan menggabungkan nilai orientasi dari sensor lain seperti IMU, sehingga dapat meminimalisis *error* akibat slip roda atau permukaan yang tidak rata. Selain itu, algoritma seperti *Extended Kalman Filter (EKF)* juga dapat digunakan untuk mengurangi akumulasi *error* yang umum terjadi pada *odometry* berbasis roda saja. Hal yang mempengaruhi *error* biasanya terjadi karena permukaan yang licin atau tidak rata, sehingga terjadi selip pada roda *encoder*.

Setiap pulsa pada *encoder* mewakili jarak tempuh tertentu, parameter lain yang menentukan perhitungan adalah keliling roda dan jumlah pulsa per putaran (PPR) pada *encoder*. Sehingga, untuk

menentukan jarak yang ditempuh oleh roda dapat menggunakan persamaan:

$$Jarak = Jumlah\ Pulsa * (\frac{Keliling\ Roda}{PPR})$$

Keterangan:

PPR = Pulsa per rotasi pada *encoder*

Parameter yang diperoleh menggunakan kinematika *odometry* tiga roda adalah: posisi robot (x, y, θ) mengacu pada kerangka robot di lingkungan yang tetap. Salah satu konfigurasi yang umum digunakan adalah tiga roda *encoder* pasif, di mana sistem ini terdiri dari tiga roda *tracking* yang tidak digerakkan oleh motor, namun berfungsi sebagai sensor gerak translasi dan rotasi. Roda-roda tersebut ditempatkan dengan konfigurasi dua roda tegak lurus satu sama lain untuk mengukur pergerakan pada sumbu X dan Y, serta satu roda yang digunakan untuk mendeteksi perubahan orientasi pada robot. Dengan pendekatan kinematika planar, *odometry* tiga roda memungkinkan estimasi posisi dalam bidang dua dimensi.

Perubahan posisi dan orientasi dihitung berdasarkan kombinasi dari perubahan posisi roda-roda tersebut. Orientasi robot (θ) dapat ditentukan dari selisih gerakan roda-roda yang sejajar dengan sumbu utama, dikaitkan dengan lebar antar roda atau *offset* posisi roda terhadap pusat rotasi robot:

$$\Delta\theta = \frac{\Delta s_{kanan} - \Delta s_{kiri}}{\text{jarak antar roda}}$$

Keterangan:

$\Delta\theta$ = Perubahan sudut orientasi robot

Δs = Perubahan jarak tempuh roda

Perubahan posisi lateral (Δy) diukur oleh roda ketiga, yang posisinya tegak lurus terhadap dua roda sebelumnya. Karena robot juga mengalami gerakan rotasi, nilai perubahan perlu dilakukan koreksi dengan mengurangi komponen gerakan akibat rotasi, sehingga didapatkan:

$$\Delta y = \Delta s_{tengah} - (\Delta\theta * d)$$

Keterangan:

Δy = Perubahan posisi lateral sumbu y

$\Delta\theta$ = Perubahan sudut orientasi robot

Δs_{tengah} = Jarak tempuh roda tengah

d = Jarak roda tengah terhadap pusat rotasi robot

Dengan d adalah jarak roda pengukur tengah terhadap pusat rotasi robot. Setelah komponen Δx , Δy dan $\Delta\theta$ dihitung, posisi robot dalam koordinat global dapat diketahui dengan menggunakan transformasi berdasarkan orientasi saat itu:

$$x_{baru} = x_{lama} + \Delta x \cdot \cos(\theta) - \Delta y \cdot \sin(\theta)$$

$$y_{baru} = y_{lama} + \Delta x \cdot \sin(\theta) + \Delta y \cdot \cos(\theta)$$

Model ini bekerja dengan mengasumsikan bahwa permukaan lintasan robot rata dan tidak terjadi selip pada roda. Pada kenyataannya, akurasi *odometry* dapat terpengaruh oleh berbagai faktor seperti selip pada roda, ketidakteraturan permukaan, atau akumulasi kesalahan dari integrasi data. Sehingga untuk mengurangi *error* yang terjadi, sistem *odometry* sering dikombinasikan dengan sensor tambahan seperti IMU dan dilengkapi dengan algoritma filter seperti *Kalman Filter* untuk mengurangi *error* secara progresif.

7. PID Controllers

Kendali PID atau PID controller yang merupakan kependekan dari (Proportional-Integral-Derrivative controller) digunakan sistem kendali untuk menghitung variabel kesalahan (*error*) antara nilai yang diinginkan (*setpoint*) dan variabel proses yang terukur (*process variable*). Hasil keluaran dari proses kendali PID digunakan untuk meminimalkan *error* tersebut. Pengendalian ini menggunakan tiga variabel kontrol utama antara lain: kontrol proporsional, integral, dan derivatif, yang diatur sedemikian rupa sehingga mampu meningkatkan stabilitas dan kinerja dalam proses otomasi.

Referensi atau target yang diinginkan disebut sebagai *setpoint* dan keluarannya disebut sebagai variabel proses terukur. Berikut merupakan konvensi penamaan variabel umum untuk jumlah yang relevan:

$$setpoint = r(t)$$

$$error = e(t)$$

$$control\ input = u(t)$$

$$output = y(t)$$

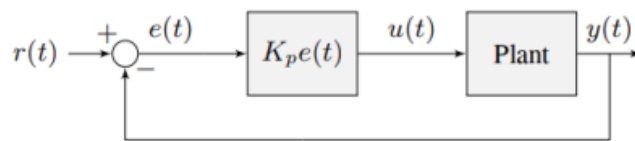
Variabel kesalahan atau *error* dapat dihitung dengan mengurangi variabel referensi dengan variabel *output*, sehingga $e(t) = r(t) - y(t)$.

a. Proportional

Variabel kontrol ini bertujuan untuk mengurangi kesalahan posisi menjadi nol. Kesalahan posisi dihitung sebagai selisih antara nilai *setpoint* dan nilai yang terukur saat ini, sehingga dapat diformulasikan sebagai berikut:

$$u(t) = k_p e(t)$$

Di mana K_p adalah gain penguatan proporsional dan $e(t)$ adalah *error* pada waktu t .



Gambar 7. Diagram Blok Kendali Proportional

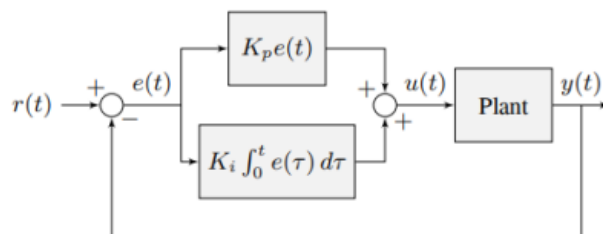
b. Integral

Variabel kontrol integral mengakumulasi kesalahan posisi dari waktu ke waktu dengan menambahkan akumulasi kesalahan ke input kendali, kontrol integral membantu mengatasi

kesalahan *steady-state* yaitu kondisi di mana kesalahan di mana sistem telah mencapai kestabilan namun keluaran tidak sepenuhnya mencapai nilai yang diinginkan. Kendali integral dapat diformulasikan menjadi:

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau$$

Di mana K_p adalah kendali penguatan proporsional, K_i adalah penguatan integral, $e(t)$ adalah eror pada waktu t , dan τ adalah variabel integrasi. $\int_0^t e(\tau) d\tau$ adalah integral dari kesalahan seiring waktu yang terakumulasi pada area di antara kurva setpoint dan keluaran.



Gambar 8. Diagram Blok Kendali Proportional-Integral

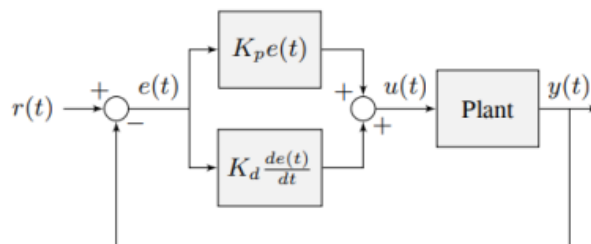
c. Derivative

Variabel kontrol derivatif bekerja dengan merespons laju perubahan kesalahan dari waktu ke waktu, kendali ini tidak hanya melihat besarnya kesalahan saat ini, tetapi juga seberapa cepat kesalahan tersebut berubah. Kendali ini bertujuan untuk mengantisipasi pergerakan sistem dan memberi *damping* agar sistem

tidak mengalami *overshoot* atau osilasi berlebih, sehingga membantu meningkatkan kestabilan dan kecepatan respons sistem. Dalam sistem kendali PID, komponen derivatif berguna dalam meredam respon kendali proporsional dan integral yang terlalu agresif. Kendali derivatif dapat diformulasikan sebagai berikut:

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau + k_d \frac{de(t)}{dt}$$

Di mana K_d adalah penguatan derivatif dan $\frac{de(t)}{dt}$ adalah turunan dari kesalahan terhadap waktu. Komponen ini memperkirakan arah dan kecepatan perubahan *error*, sehingga sistem dapat mengambil tindakan korektif sebelum kesalahan menjadi lebih besar, sehingga dapat dikatakan bahwa kendali derivatif bertindak sebagai prediktor terhadap kesalahan untuk memberikan respons kendali yang lebih halus dan stabil.

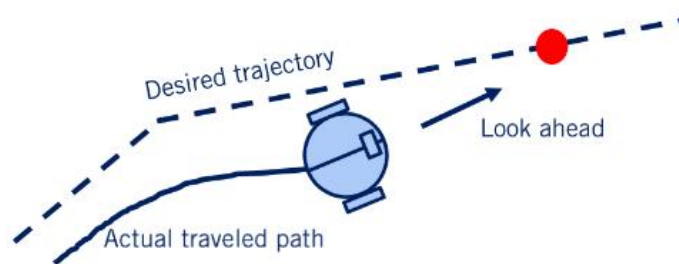


Gambar 9. Diagram Blok Kendali Proportional-Derivative

8. Pure Pursuit

Pure Pursuit merupakan sebuah algoritma *path following*. Algoritma ini bekerja dengan cara menghitung kecepatan angular (θ) yang

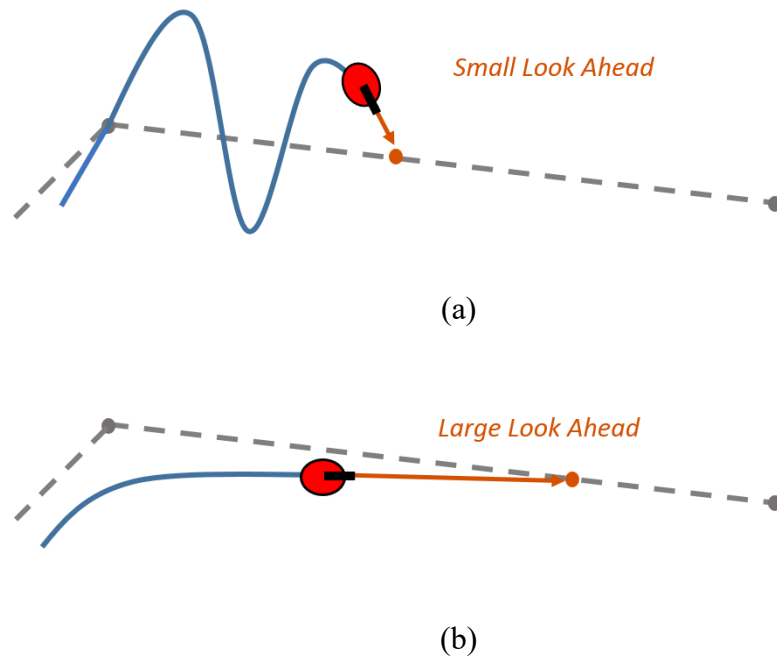
menggerakkan robot dari posisi saat ini menuju ke sebuah titik yang dinamakan *look-ahead* yang berada di depan robot. Kecepatan linear robot diasumsikan konstan, sehingga dapat diubah kapanpun. Algoritma ini berusaha untuk mengejar *look-ahead* agar robot dapat mengikuti jalur.



Gambar 10. Algoritma Pure Pursuit
(Sumber: <https://medium.com>)

Terdapat parameter yang dapat diubah yaitu seberapa jauh titik *look-ahead* tersebut. *Input waypoint* berupa koordinat dalam sumbu $[x, y]$, yang kemudian digunakan untuk menghitung kecepatan robot. *Pose* awal robot juga dimasukkan sebagai pengukuran dalam radian mulai dari 0° berlawanan dengan arah jarum jam. *Look Ahead distance* merupakan seberapa jauh pada *path* robot harus melihat ke depan untuk menghitung seberapa jauh titik *look-ahead* ditempatkan.

Nilai dari *look ahead* mempengaruhi pergerakan robot, apabila nilainya terlalu kecil akan membuat robot menjadi osilasi saat bergerak sesuai jalur, dan apabila terlalu besar akan membuat *path* yang dilalui robot menjadi terlalu menyimpang.



Gambar 11. Pengaruh Parameter Look Ahead Distance (a) kecil (b) besar

B. Komponen Perangkat Lunak (*Software*)

1. Simulasi Kinematika Robot Berbasis Unity



Gambar 12. Software Unity 3D
(Sumber: <https://1000logos.net/unity-logo/>)

Unity merupakan *platform* pengembangan 3D yang dapat digunakan untuk mensimulasikan kinematika robotika. Unity menyediakan fitur visualisasi, pemodelan fisika, dan scripting untuk membuat simulasi kinematika robot secara realistis. Unity 3D sebenarnya adalah perangkat lunak *game engine* untuk membangun permainan 3D dikembangkan

oleh *Unity Technologies*. Pada awal pengembangan Unity 3D hanya berfokus ke pengembangan untuk kebutuhan di industri permainan, namun saat ini telah banyak dimanfaatkan dalam berbagai bidang, khususnya termasuk di bidang robotika. Unity 3D dapat merepresentasikan gerakan robot berdasarkan model kinematika, baik kinematika maju (*forward kinematics*) maupun kinematika balik (*inverse kinematics*). Simulasi berbasis Unity memberikan keunggulan, seperti kemudahan integrasi dengan sensor virtual, pengujian algoritma navigasi, dan visualisasi respon sistem terhadap berbagai skenario. Melalui simulasi kinerja robot dapat diamati tanpa adanya risiko kerusakan perangkat keras. Penggunaan Unity sebagai simulator di bidang robotika berfungsi sebagai visualisasi dan sarana pembelajaran interaktif, sehingga peserta didik atau peneliti dapat menguji parameter kontrol PID, memvisualisasikan gerakan robot secara fleksibel.

Simulator telah digunakan di berbagai bidang pembelajaran termasuk robotik, media pembelajaran ini telah banyak digunakan sebagai alternatif dalam pembelajaran robotik yang memberikan pengetahuan melalui gamifikasi (Tselegkaridis & Sapounidis, 2021). Selain menggunakan robot fisik sebagai media pembelajaran, simulasi robotika berbasis perangkat lunak memungkinkan fleksibilitas dan efektivitas lebih, karena peserta didik dapat mempelajari robotika di mana saja dan kapan saja selama memiliki perangkat yang mendukung. Meskipun sulit untuk merepresentasikan simulasi dengan kondisi di

lingkungan nyata, simulator tetap menjadi pilihan yang tepat ketika tidak tersedia robot fisik. Adapun fitur-fitur yang dimiliki oleh Unity 3D untuk mengembangkan simulasi antara lain sebagai berikut:

- a. *Integrated Development Environment* (IDE) atau lingkungan pengembangan terpadu.
- b. *Graphic Engine* menggunakan Direct3D (Windows), OpenGL ES (iOS), dan *proprietary* API.
- c. *Game Scripting* melalui *framework* Mono yang merupakan implementasi dari .NET *framework* berbasis bahasa C#.

Dalam pengembangan simulasi robot berbasis Unity, fitur scripting memungkinkan pengembang untuk mengimplementasikan model kinematika secara langsung ke dalam gerakan visual objek 3D, dengan cara menghitung dan menerjemahkan parameter kinematika seperti kecepatan roda dan sudut orientasi roda ke dalam translasi serta rotasi objek robot secara *realtime*. Simulasi *forward kinematics* dilakukan dengan memproyeksikan komponen kecepatan masing-masing roda ke arah sumbu X dan sumbu Y menggunakan fungsi trigonometri, berdasarkan orientasi setiap modul swerve. Kemudian, nilai kecepatan translasi total dihitung sebagai rata-rata dari semua vektor kecepatan roda, lalu dikalikan dengan waktu *frame* (Time.deltaTime) di Unity untuk mensimulasikan perpindahan posisi secara kontinu. Unity menyediakan metode transform.Translate untuk menerapkan perpindahan objek dan transform.Rotate untuk melakukan rotasi

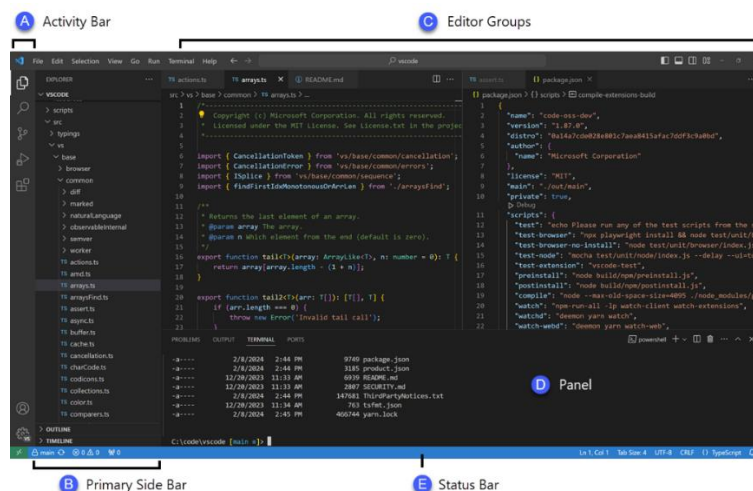
terhadap sumbu orientasi robot, berdasarkan nilai kecepatan sudut (θ) yang dihitung dari distribusi kecepatan antar roda. Selain simulasi *forward kinematics* dapat juga diimplementasikan *inverse kinematics* untuk menghitung parameter aktuasi seperti arah sudut dan kecepatan roda berdasarkan gerakan yang diinginkan, seperti translasi maju-mundur, menyamping (*strafe*), maupun rotasi. Pada sistem *swerve drive*, *inverse kinematics* menjadi inti dari pengendalian karena setiap modul harus diarahkan pada sudut tertentu dan diputar dengan kecepatan yang sesuai agar seluruh robot dapat bergerak secara terkoordinasi. Dalam penerapannya *inverse kinematics* dihitung dengan menggunakan persamaan geometris dan trigonometri untuk mendapatkan arah dan kecepatan masing-masing roda berdasarkan input dari pengguna. Perhitungan melibatkan transformasi koordinat berdasarkan sudut orientasi robot terhadap dunia, kemudian menghitung vektor kecepatan untuk setiap roda berdasarkan kontribusi translasi (maju/mundur dan menyamping) serta rotasi terhadap pusat robot.

2. Visual Studio Code

Visual Studio Code biasa disebut juga dengan VS Code adalah editor kode *multi-platform* yang mendukung berbagai bahasa pemrograman termasuk Python dan *framework* melalui ekosistem ekstensi yang luas serta arsitektur modular berbasis layanan. VS Code digunakan dalam pengembangan perangkat lunak profesional terutama karena memiliki berbagai fitur seperti *debugging*, *IntelliSense* untuk *auto-completion*,

integrasi dengan Git, dan kemampuan penyesuaian melalui ekstensi yang dapat memperkaya pengalaman pemrograman. Dengan VS Code, pengguna dapat menulis, menjalankan, dan *debug* kode Python secara langsung di dalam editor, termasuk bekerja dengan Python Interactive Window yang memungkinkan kombinasi kode dan dokumentasi interaktif.

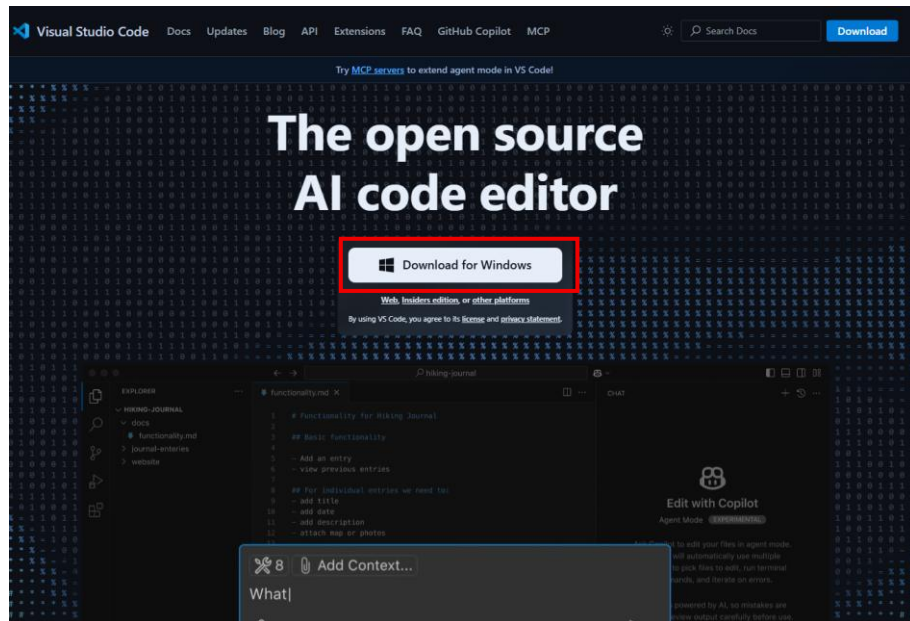
VS Code juga mendukung banyak bahasa pemrograman lain seperti JavaScript, Java, C/C++, C#, dan *framework* mobile seperti React Native dan Flutter, serta memungkinkan pengembangan aplikasi web. VS Code telah dievaluasi unggul dalam segi fungsionalitas, keandalan, portabilitas, dan efisiensi, sehingga banyak penelitian dan pengembangan mengadopsi metode ADDIE dan R&D untuk membangun media pembelajaran dan aplikasi edukasi melalui VS Code secara sistematis, menjadikannya alat utama dalam pendidikan dan riset teknologi perangkat lunak modern.



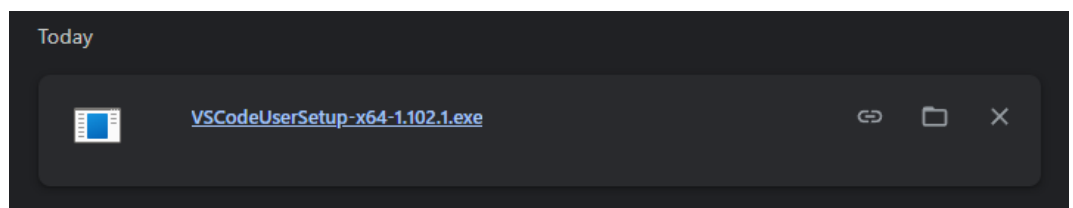
Gambar 13 Antarmuka Visual Studio Code
(Sumber: <https://code.visualstudio.com/>)

C. Instalasi Visual Studio Code

1. Buka laman berikut <https://code.visualstudio.com/>.
2. Klik Download for Windows, sesuaikan dengan sistem operasi yang digunakan.



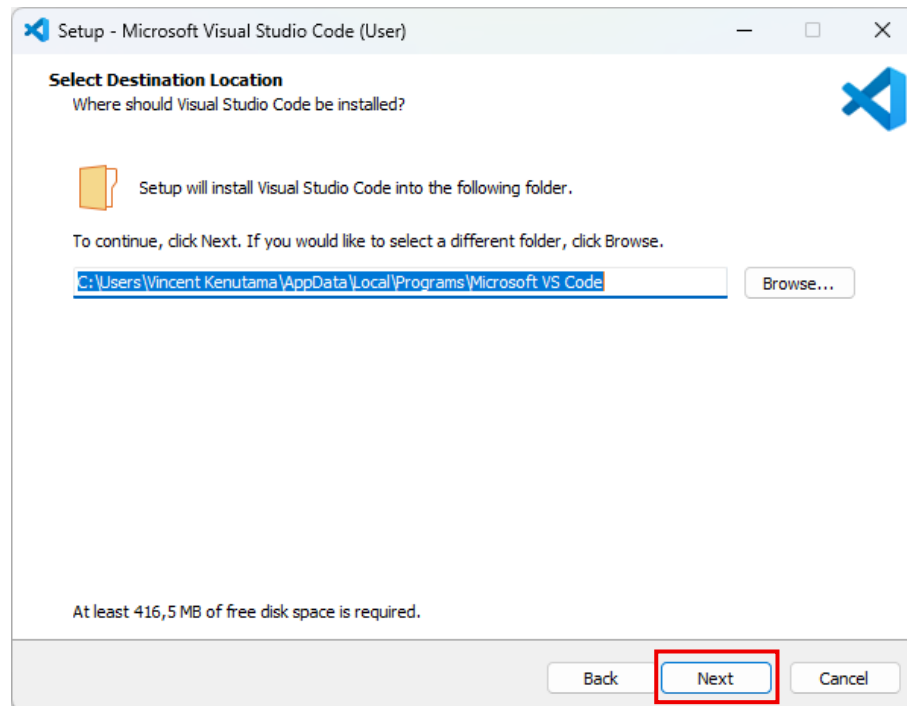
3. Buka file instalasi untuk Visual Studio Code.



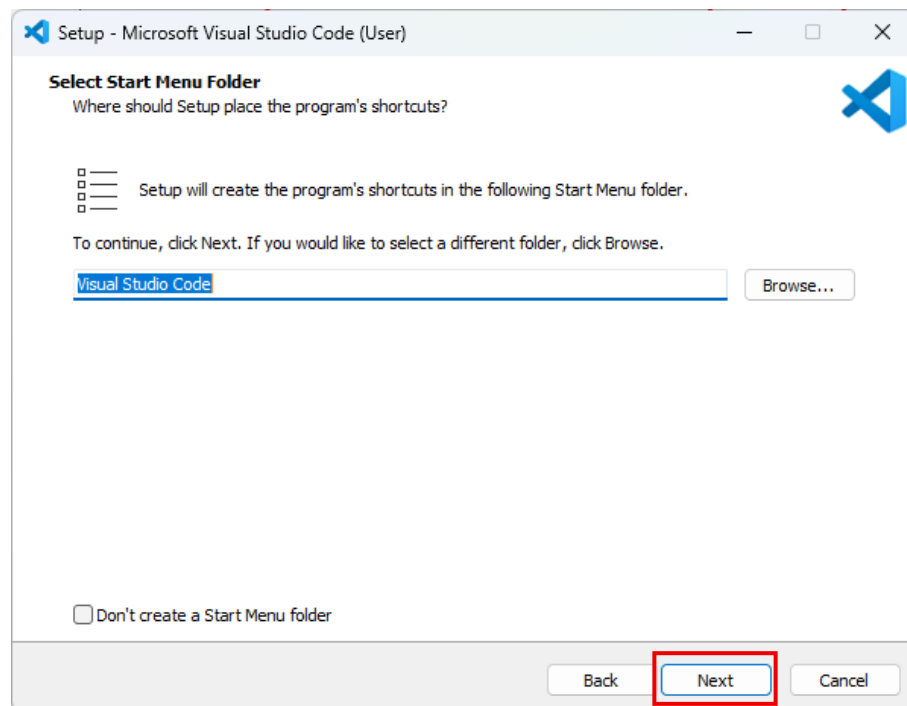
4. Pada *license agreement* klik pada *checkbox* “I accept the agreement” lalu klik tombol next.



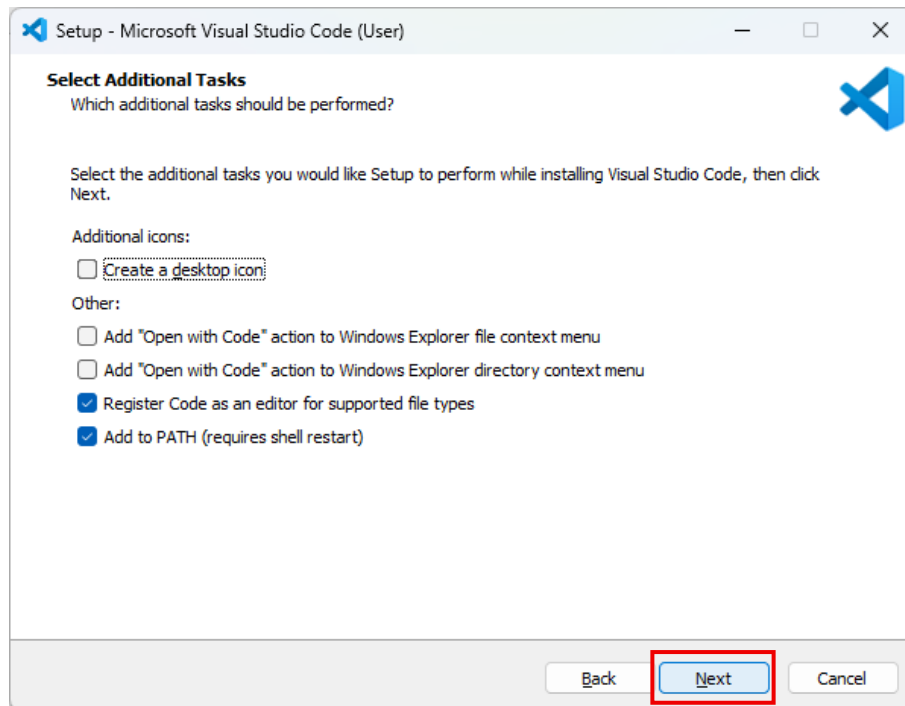
5. Pilih lokasi untuk menyimpan data program Visual Studio Code, jika tidak ingin mengubah lokasi *default*, lanjutkan dengan klik next.



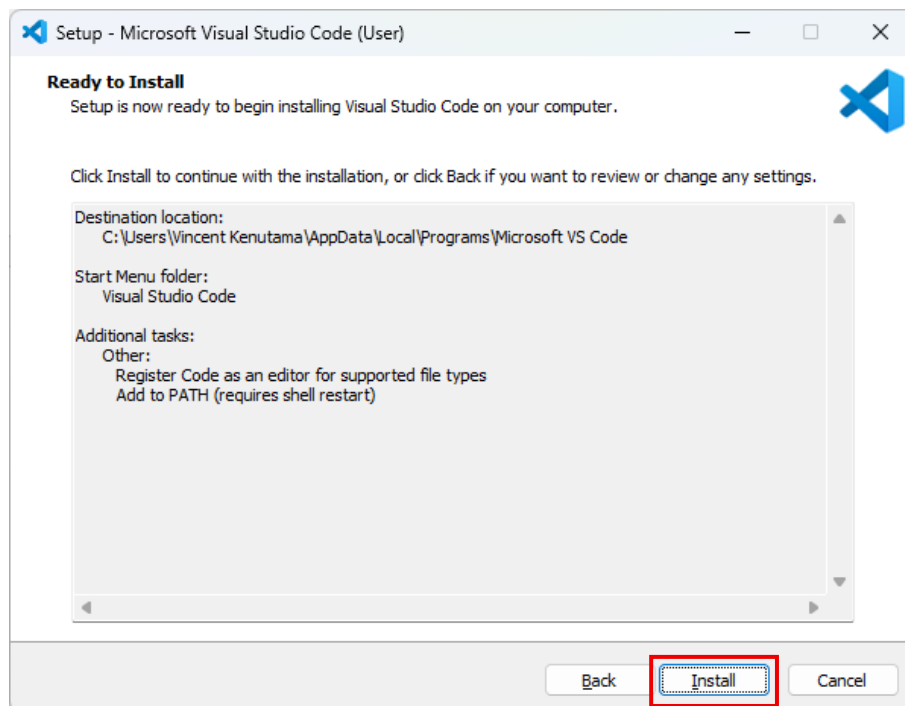
6. Klik next.



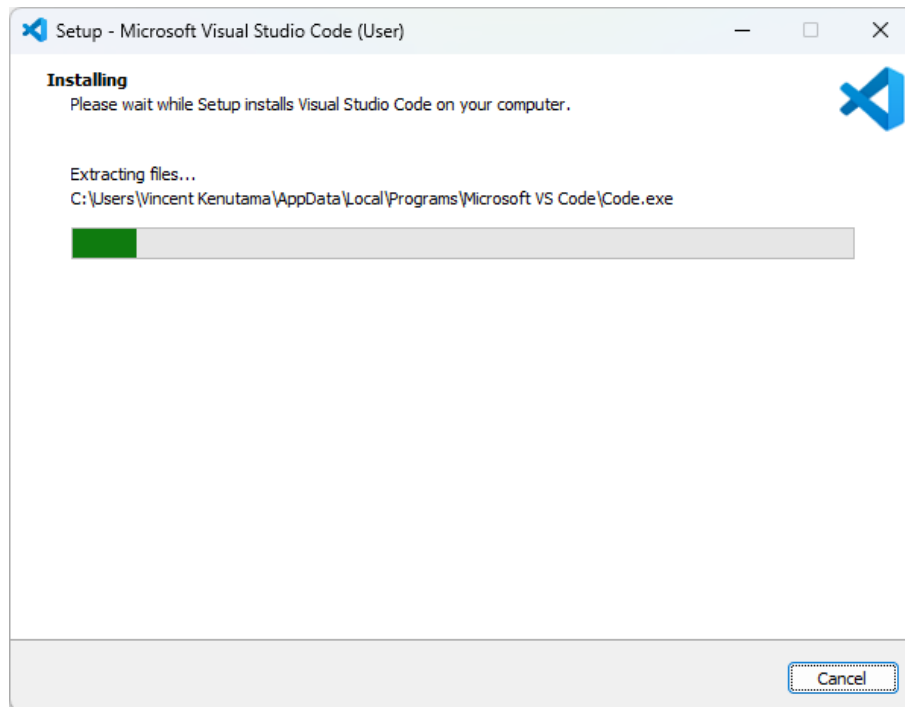
7. Lanjutkan klik next.



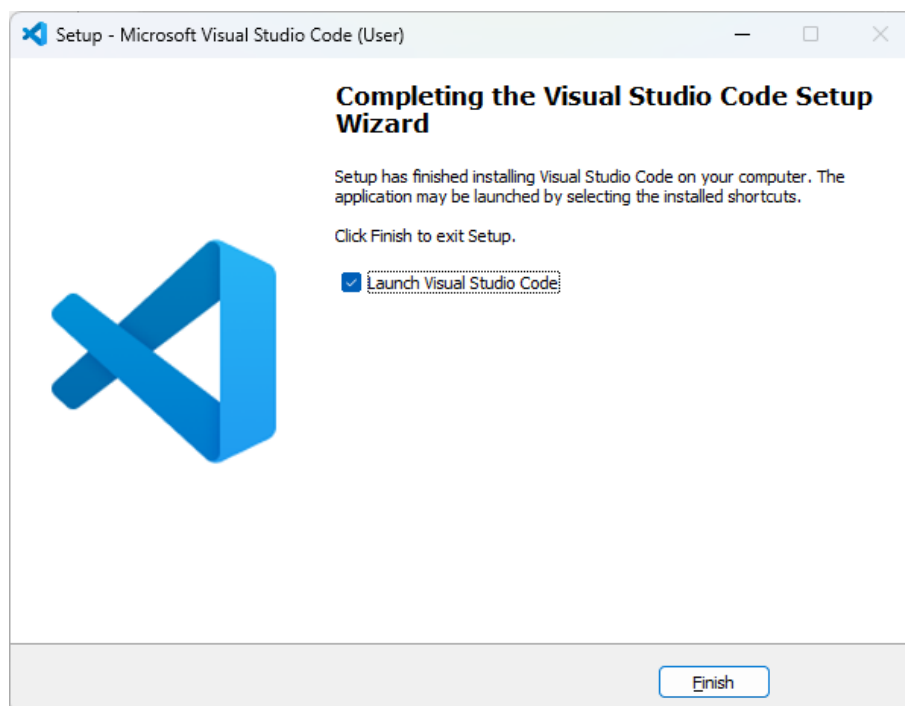
8. Kemudian klik *Install* untuk memulai prosedur instalasi program VS Code.



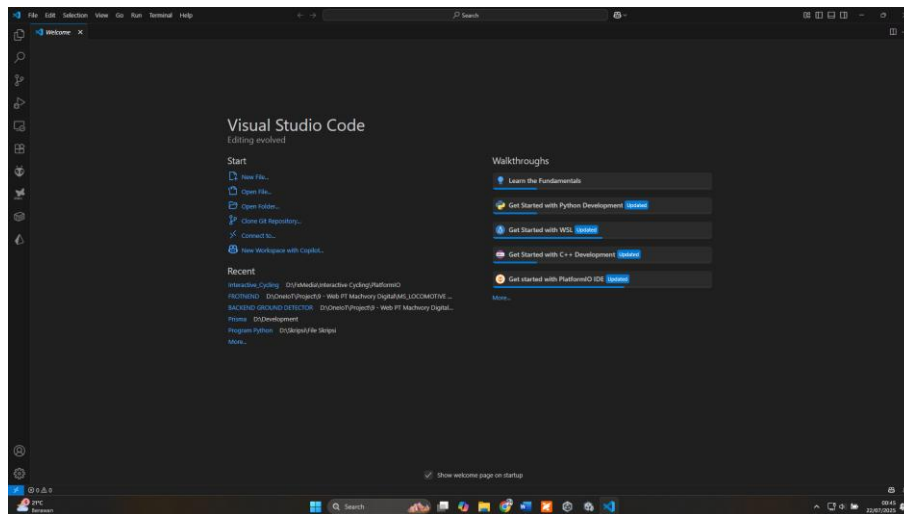
9. Tunggu hingga proses instalasi selesai.



10. Klik tombol *Finish* untuk melanjutkan membuka VS Code.



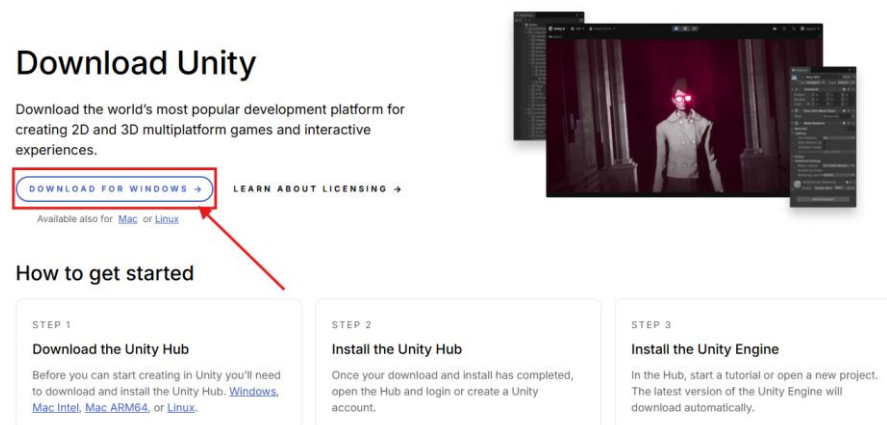
11. Berikut merupakan tampilan awal VS Code.



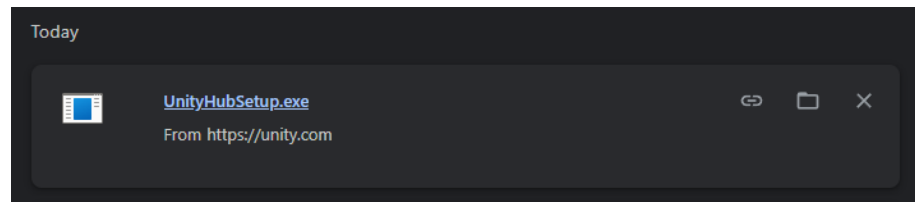
D. Langkah Persiapan Pembelajaran

Instalasi *Software* Unity 3D

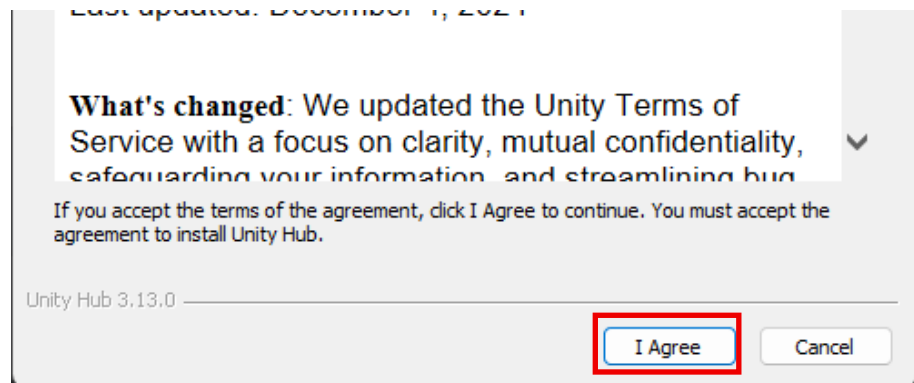
1. Unduh Unity 3D di laman <https://unity.com/download>, sesuaikan dengan sistem operasi yang digunakan.



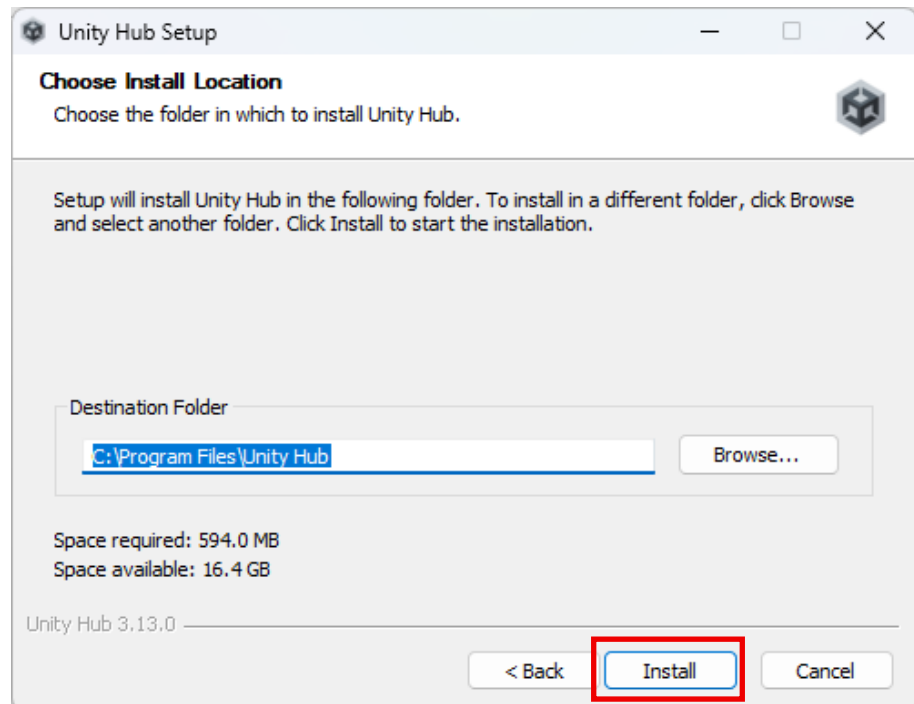
2. Buka installer Unity Hub yang telah diunduh sebelumnya, untuk memulai pemasangan Unity 3D.



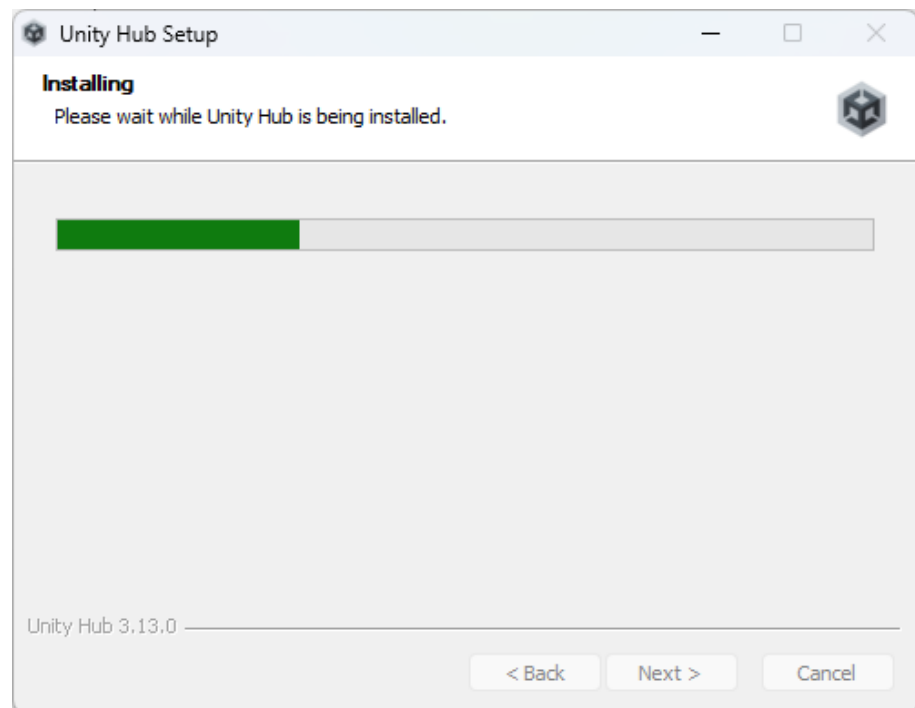
3. Setelah terbuka, pada halaman pertama akan menampilkan *Terms and Service*, klik I Agree untuk melanjutkan.



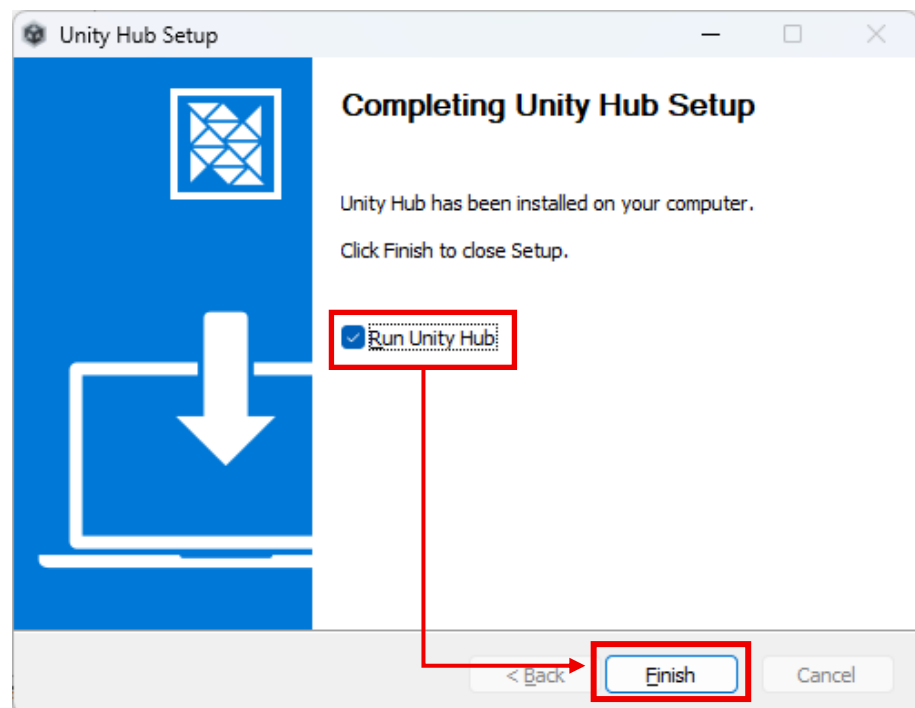
4. Pilih lokasi penyimpanan program Unity 3D, jika ingin menggunakan lokasi *default*, maka klik Install.



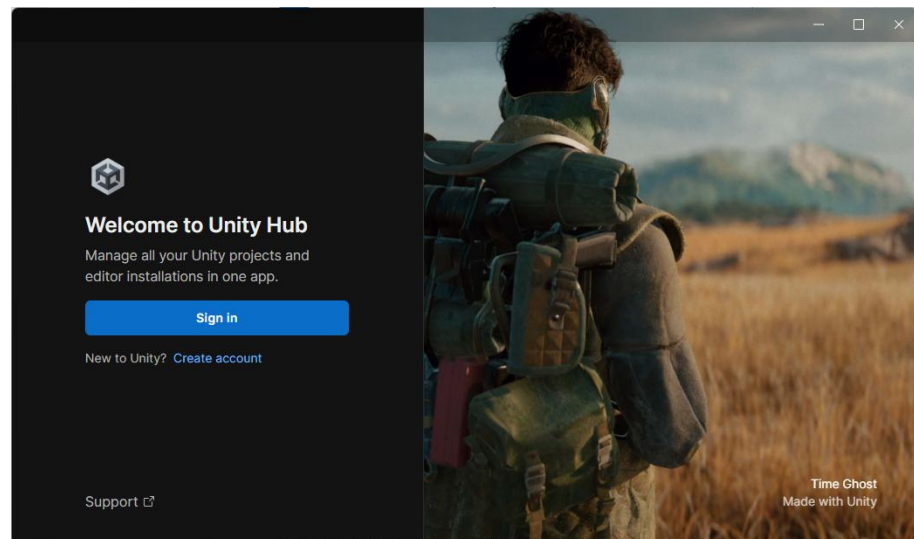
5. Tunggu proses instalasi sampai selesai.



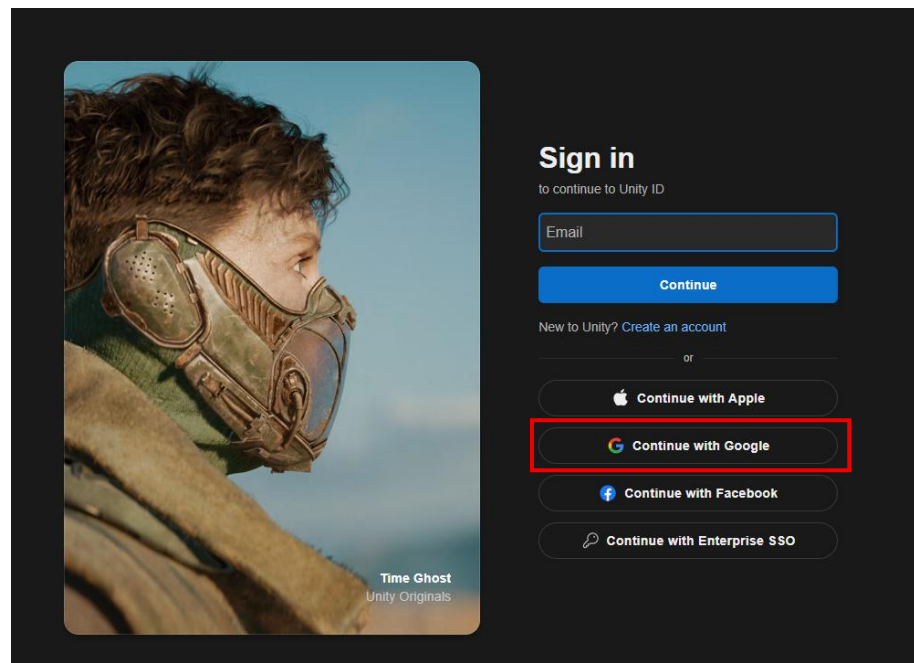
6. Setelah proses instalasi selesai, untuk menjalankan Unity Hub untuk pertama kali, klik finish dengan “Run Unity Hub” tercentang.



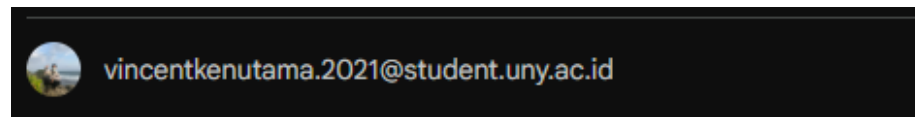
7. Setelah Unity Hub terbuka maka, Unity Hub akan meminta untuk login terlebih dahulu, apabila belum ada akun Unity, klik pada Create Account, sedangkan jika telah memiliki akun dapat klik pada Sign In.



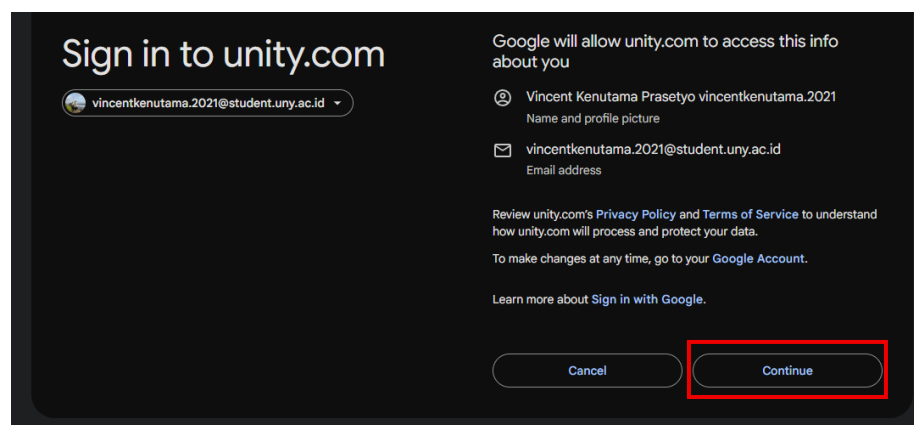
8. Gunakan akun google untuk melanjutkan pendaftaran.



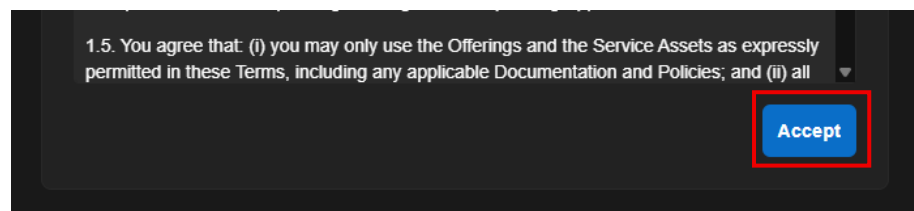
9. Jika telah login akun SSO UNY, maka pilih akun tersebut untuk melanjutkan, apabila belum login terlebih dahulu ke gmail menggunakan email UNY.



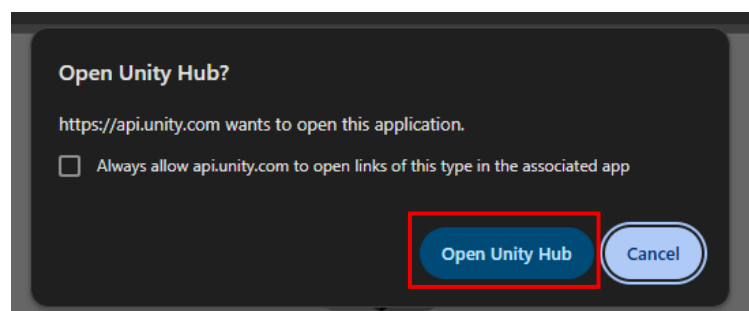
10. Klik continue untuk melanjutkan.



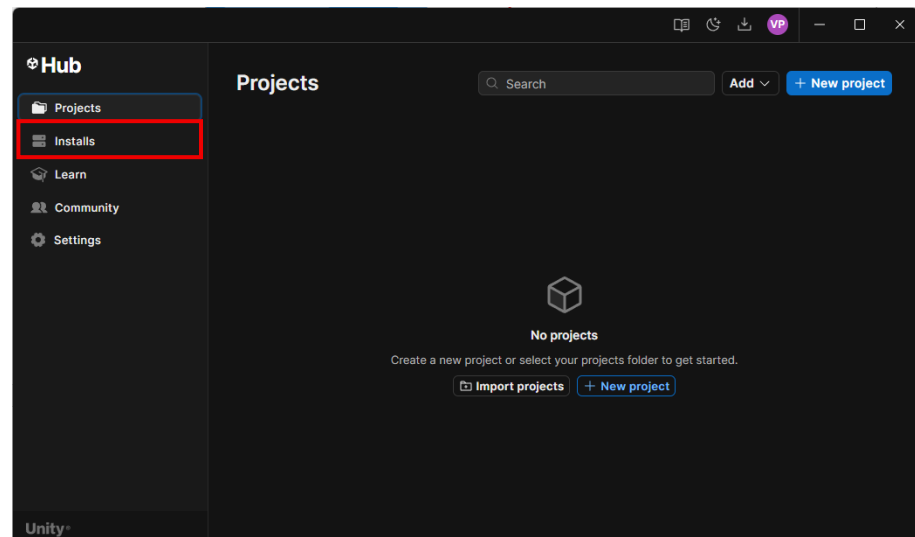
11. Klik accept.



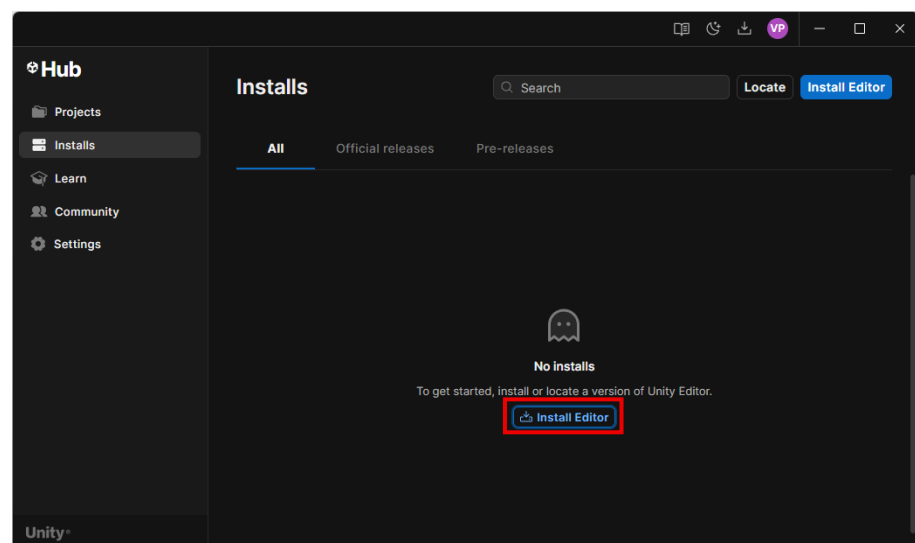
12. Jika sudah, kembali ke Unity Hub lalu klik Sign in, kemudian akan dibawa ke browser dan klik Open Unity Hub.



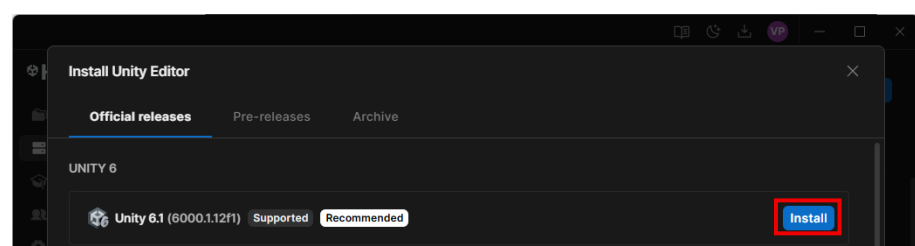
13. Pada tampilan awal Unity Hub beralih ke tab Installs untuk memulai instalasi Unity Editor.



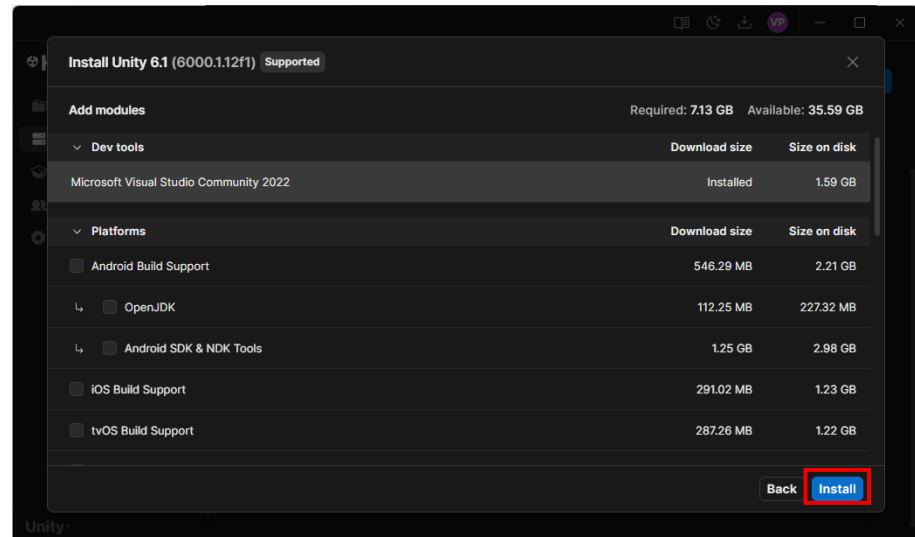
14. Klik pada Install Editor.



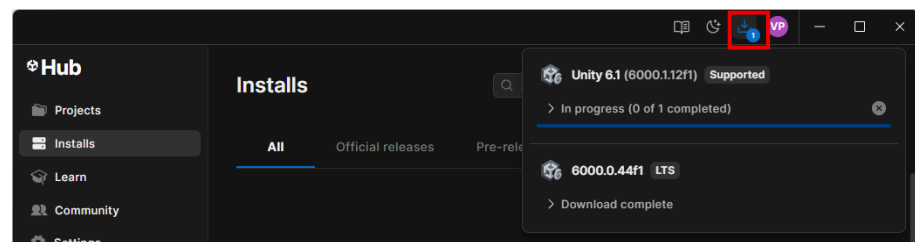
15. Pilih versi terbaru, lalu klik install.



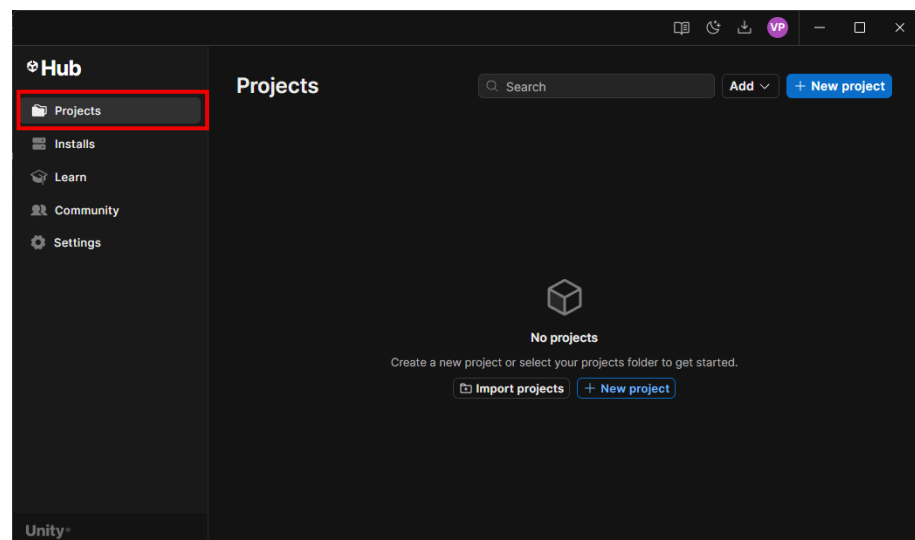
16. Jika muncul tampilan modules, klik Install untuk memulai instalasi Unity Editor.



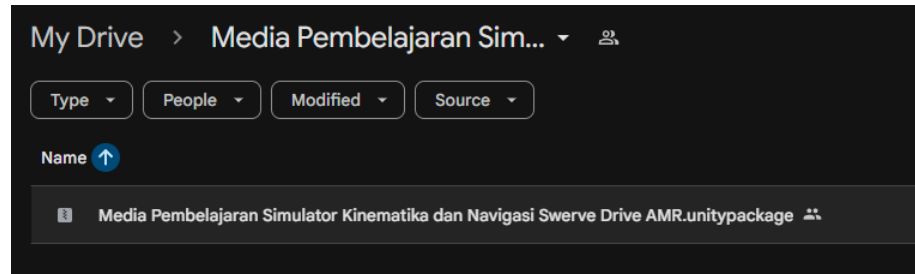
17. Tunggu proses pengunduhan dan instalasi Unity Editor sampai selesai.



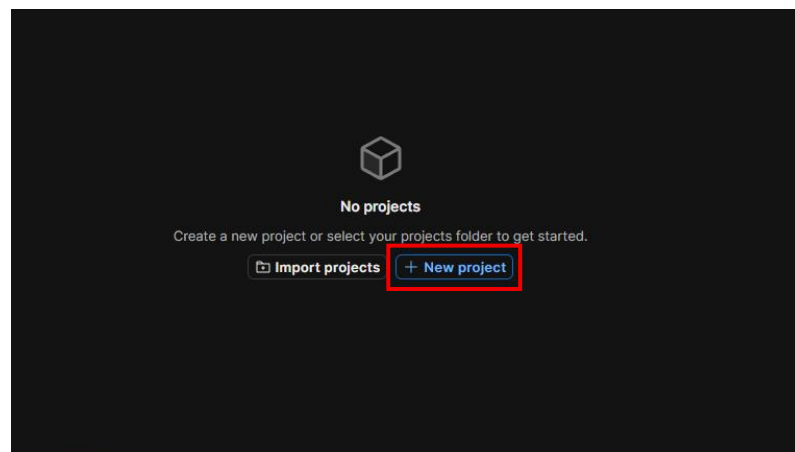
18. Jika unduhan dan instalasi telah selesai maka, kembali ke tab projects.



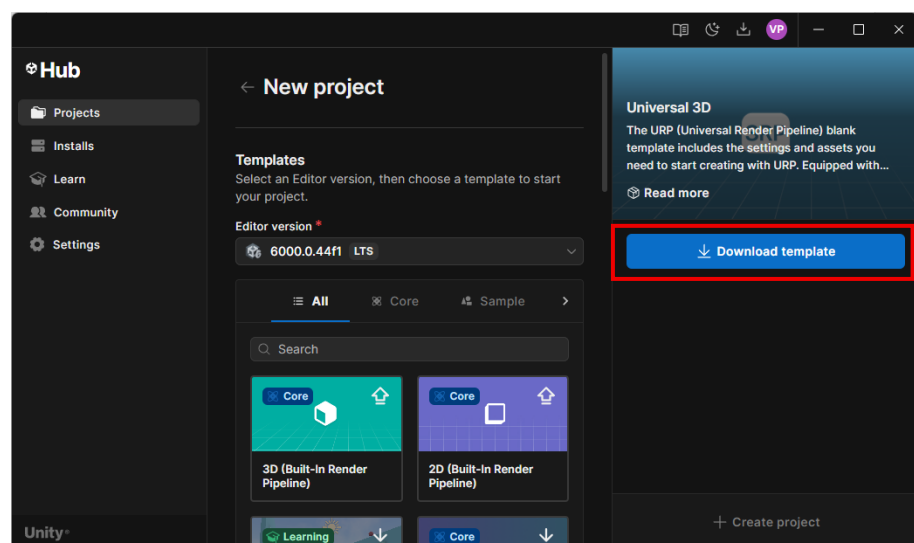
19. Unduh .unitypackage pada link <http://bit.ly/3GU5oDn>, kemudian unduh file seperti pada gambar di bawah ini.



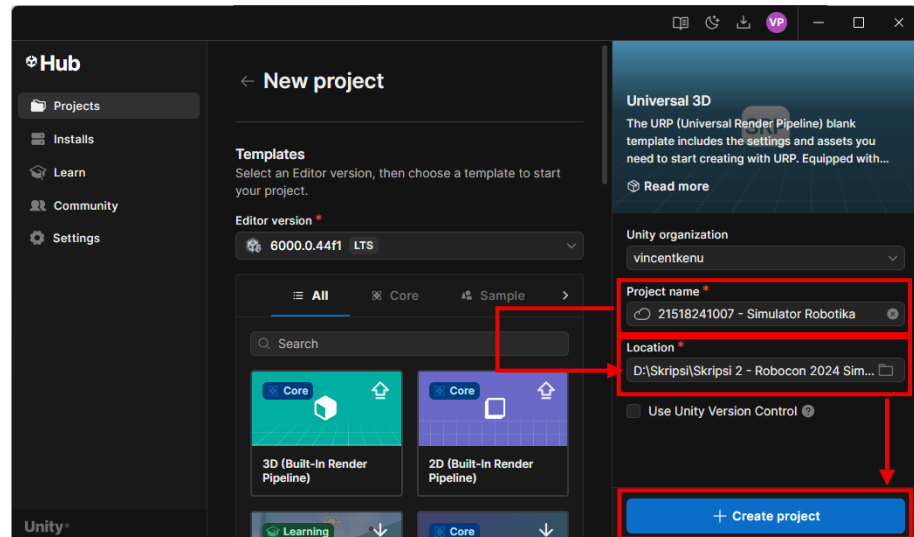
20. Setelah selesai mengunduh file simulator, beralih ke Unity Hub lalu klik New Project.



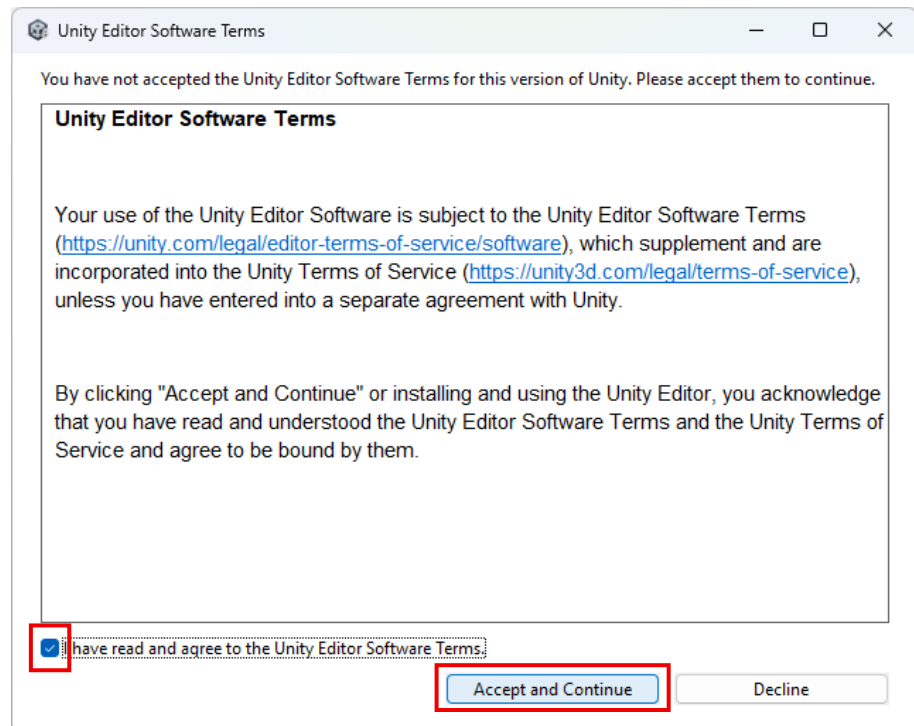
21. Klik Download Template pada Universal 3D.



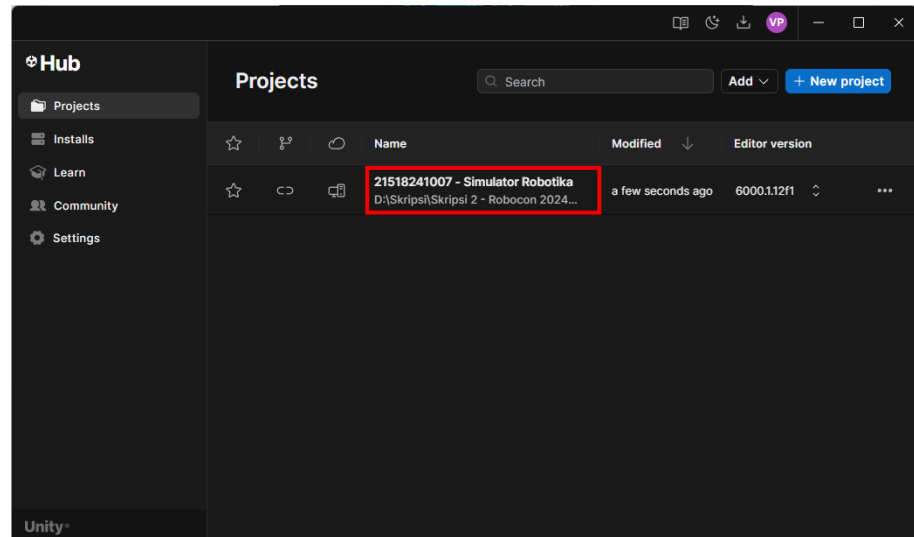
22. Berikan nama project “NIM - Simulator Robotika”, lalu pilih lokasi tempat penyimpanan project, setelah selesai klik pada Create Project.



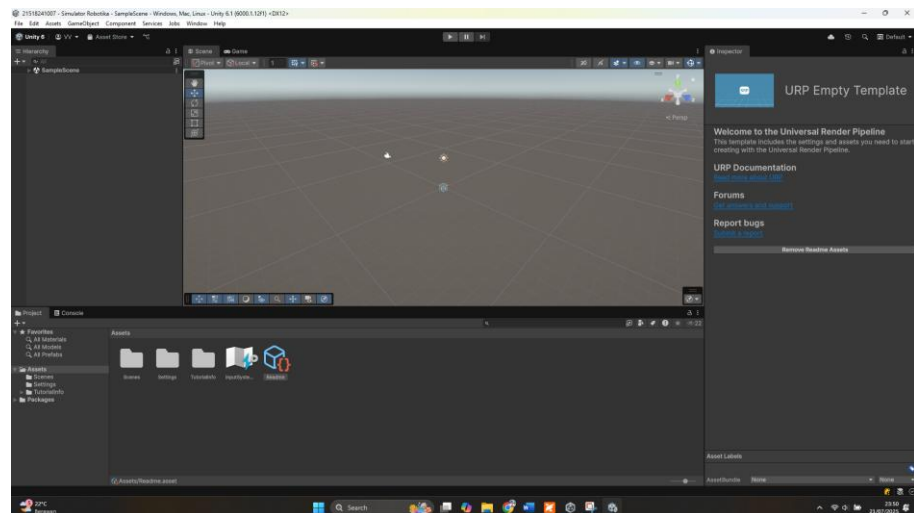
23. Jika muncul tampilan Unity Editor Software Terms, klik pada checkbox “I have read and agree to the Unity Editor Software Terms”, kemudian klik Accept and Continue.



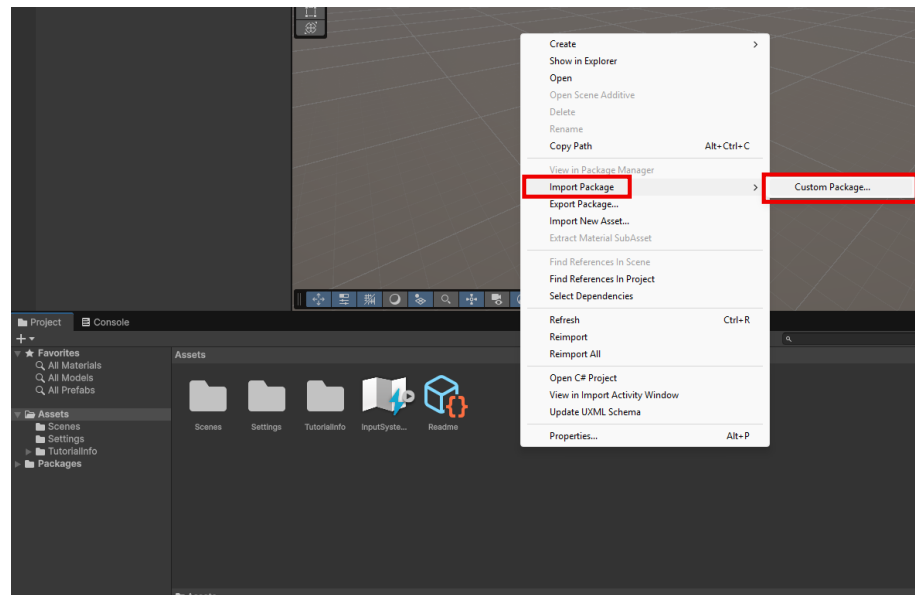
24. Untuk memulai Unity Editor klik pada nama project seperti pada gambar di bawah ini, yang telah dibuat sebelumnya.



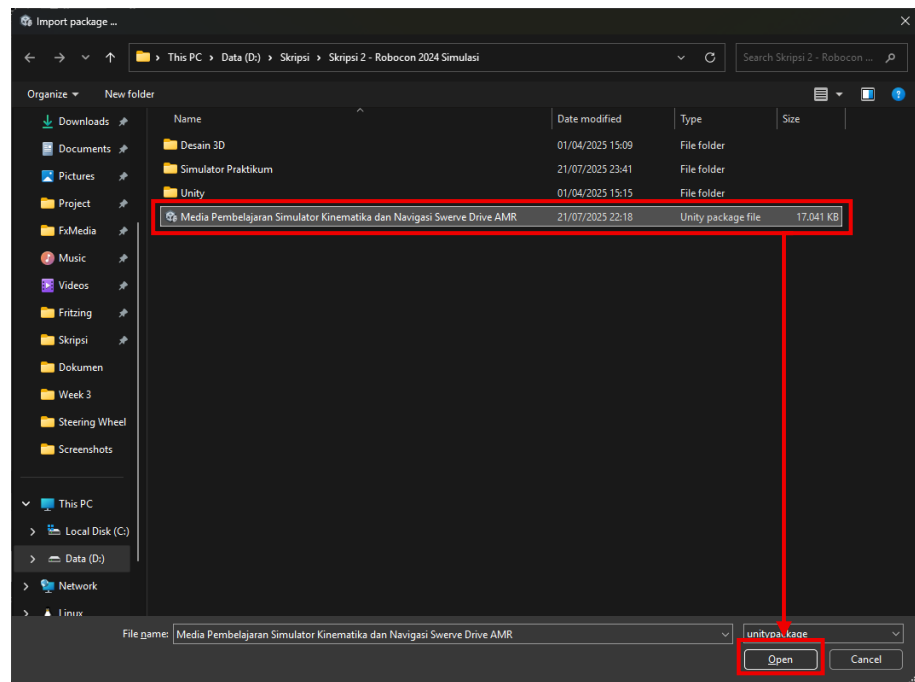
25. Setelah selesai membuka project, berikut merupakan tampilan awal dari Unity Editor.



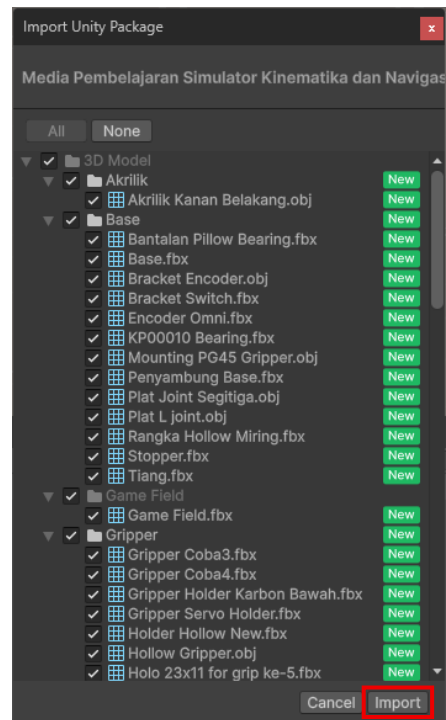
26. Klik kanan pada bagian kosong di *Assets* yang memiliki banyak folder, kemudian *Import Package > Custom Package*.



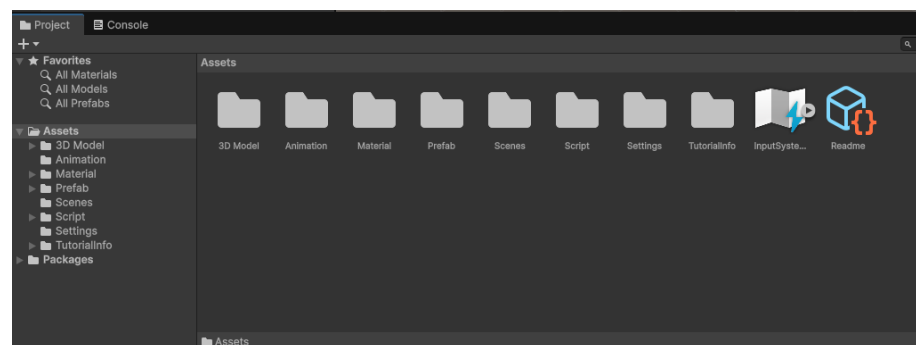
27. Cari folder tempat penyimpanan file dengan ekstensi .unitypackage yang sebelumnya telah diunduh, kemudian pilih file dan klik Open.



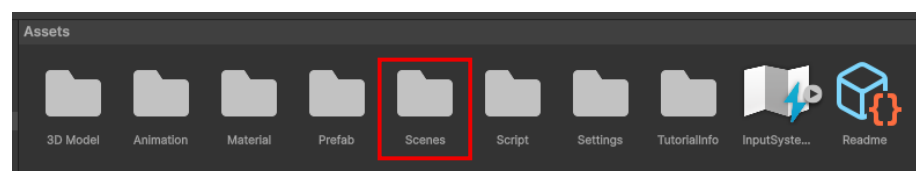
28. Pada tampilan Import Unity Package klik Import.



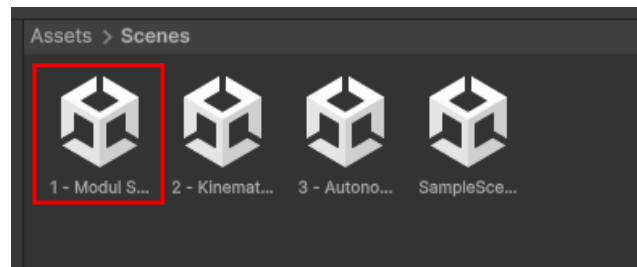
29. Setelah proses *import* selesai, maka folder *Assets* akan memiliki banyak folder.



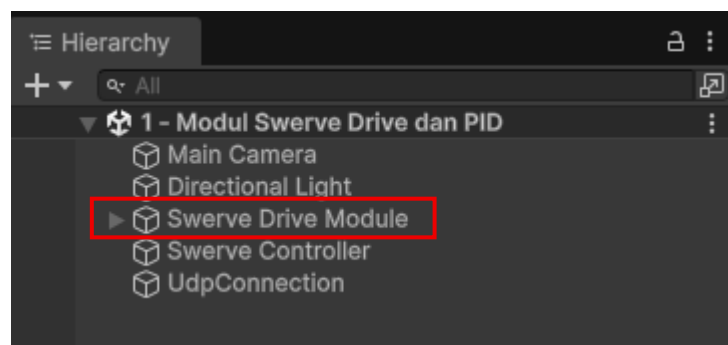
30. *Double* klik pada folder *Scenes*.



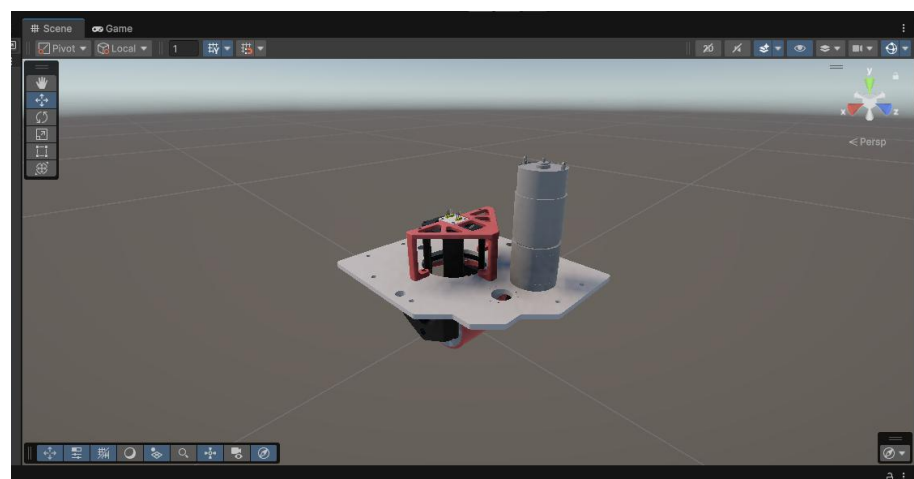
31. Buka scene “1 – Modul Swerve Drive dan PID” dengan *double* klik.



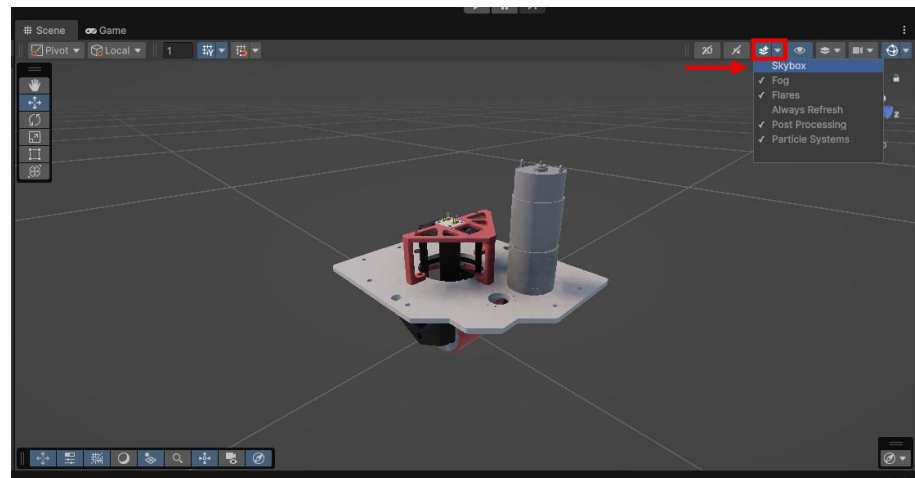
32. Jika terlalu jauh dari *3D object* modul *Swerve Drive*, pada bagian Hierarchy *double* klik pada Game Object “Swerve Drive Module”.



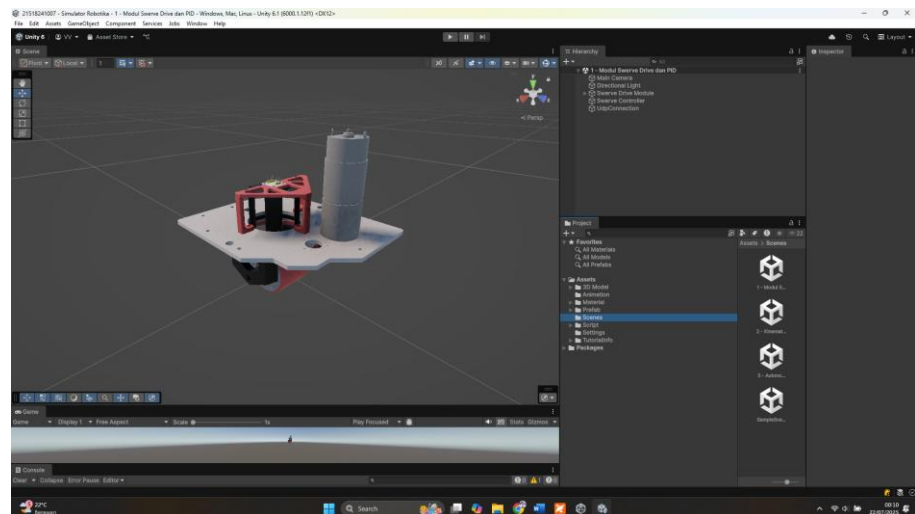
33. Maka di bagian Scene akan menampilkan visualisasi tiga dimensi dari modul *Swerve Drive*.



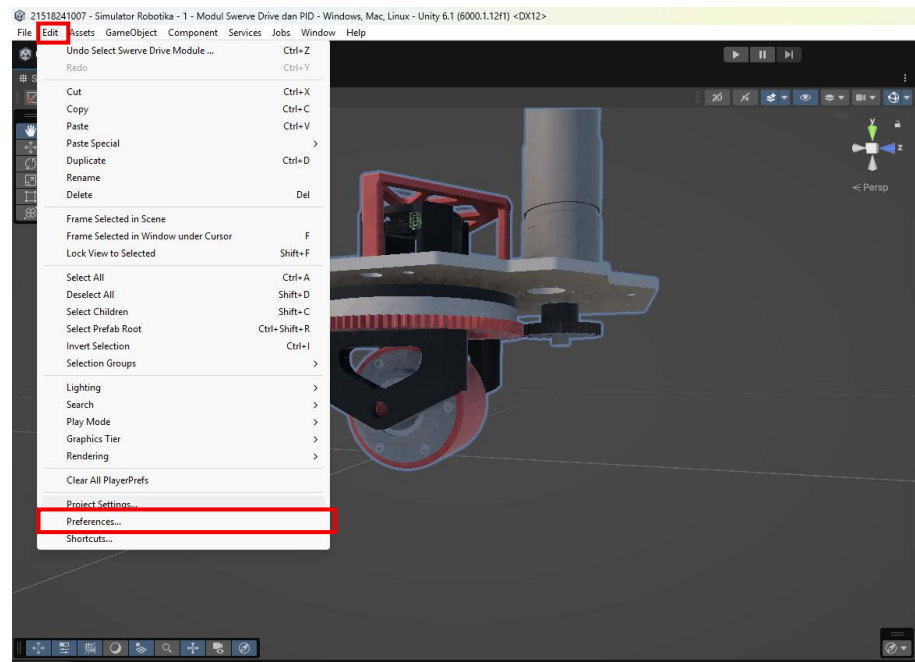
34. Matikan skybox agar tampilan pada Scene lebih berfokus pada module *Swerve Drive*.



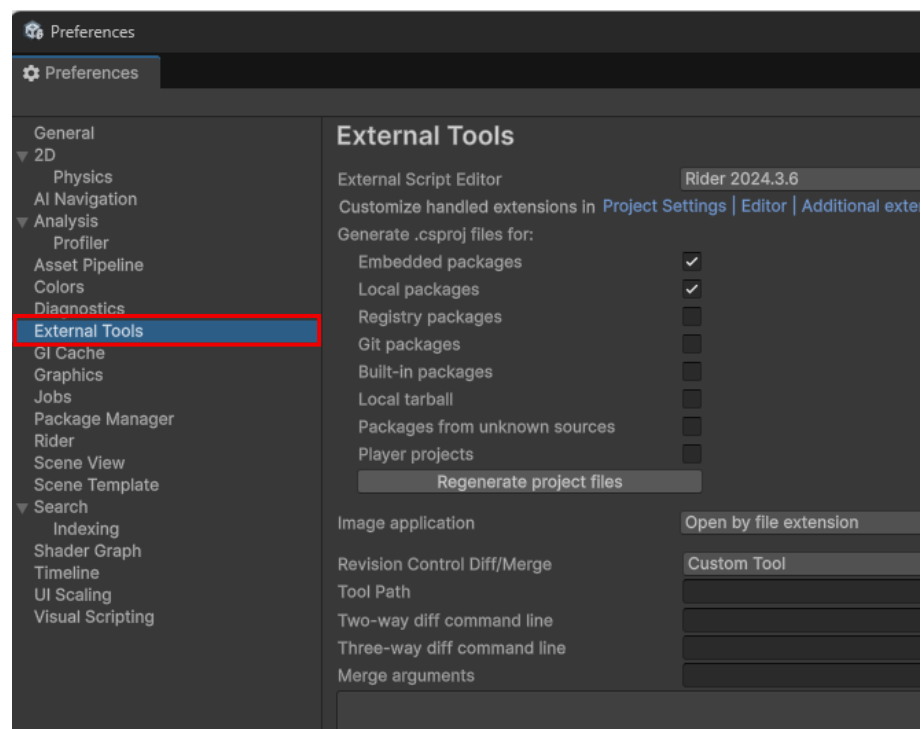
35. Coba untuk membuat layout pada Unity Editor menjadi seperti gambar di bawah ini dengan cara *drag and drop* pada setiap bagian *window*.



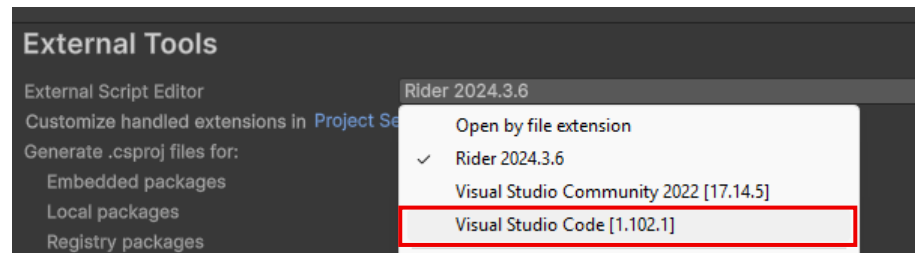
36. Konfigurasi untuk code editor yang akan digunakan, klik pada menu *Edit > Preferences*.



37. Klik pada tab *External Tools*.

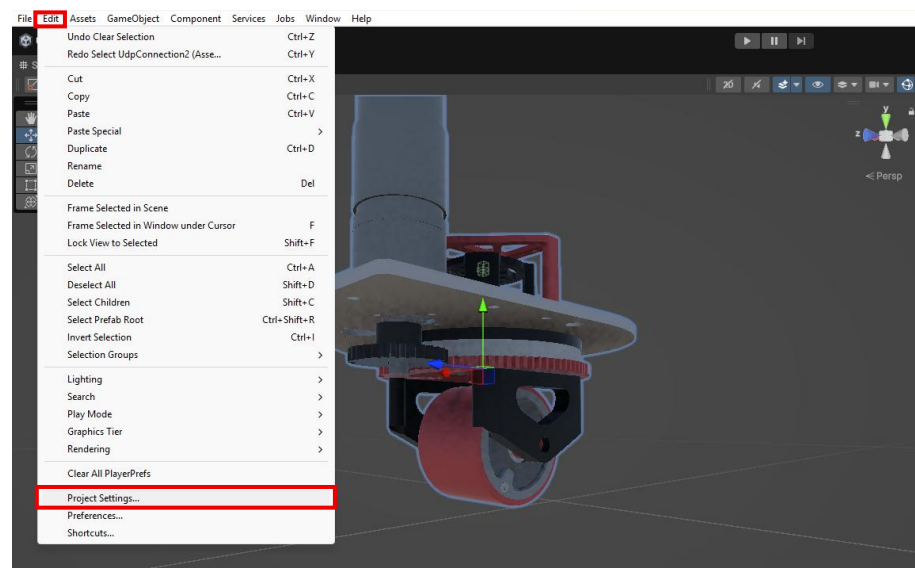


38. Pada External Script Editor ganti menjadi Visual Studio Code.

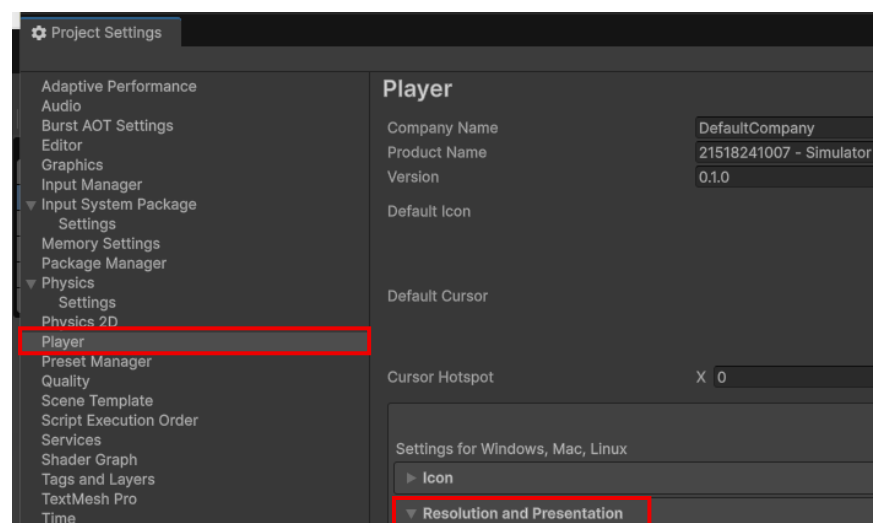


39. Tutup jendela *preferences*.

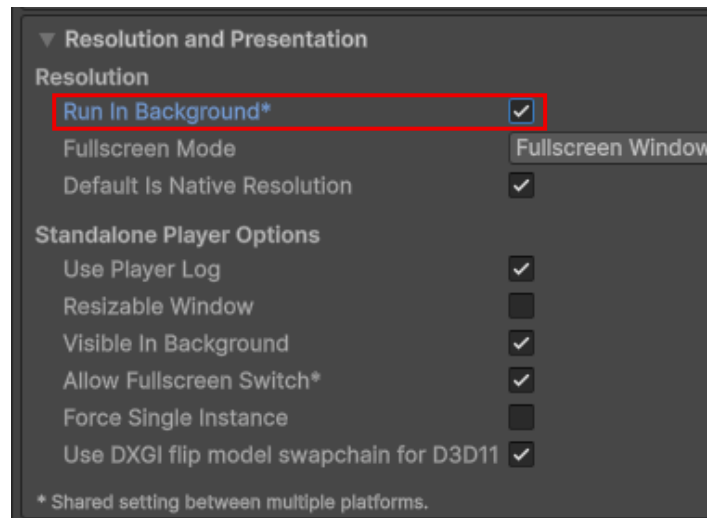
40. Navigasi ke *Edit > Project Settings*.



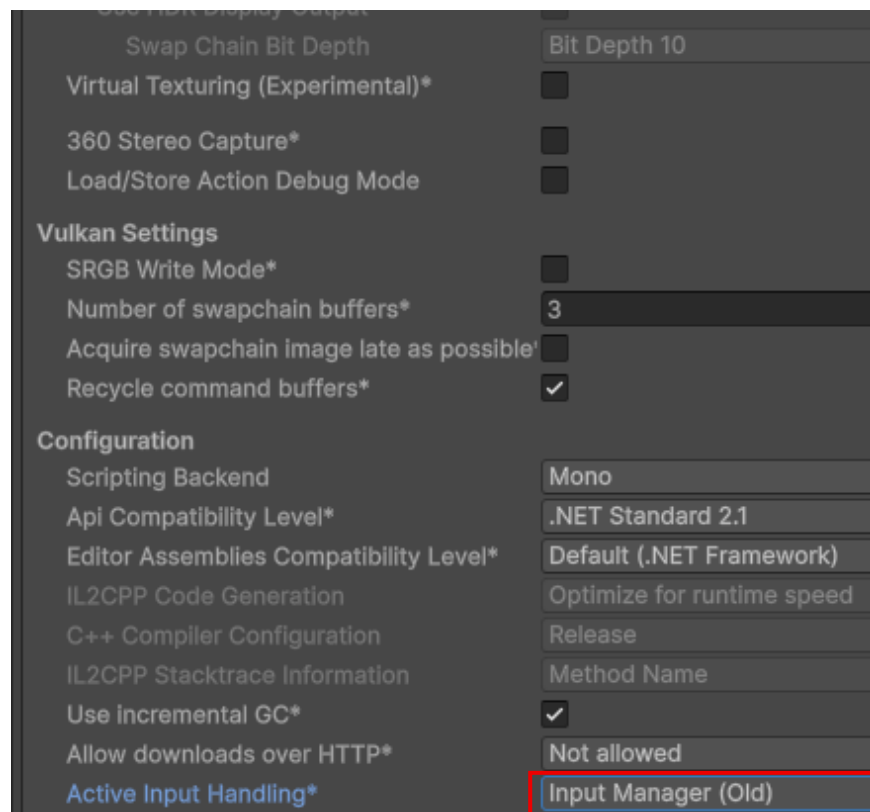
41. Pada menu tab pilih *Player*, lalu *expand* bagian *Resolution and Presentation*.



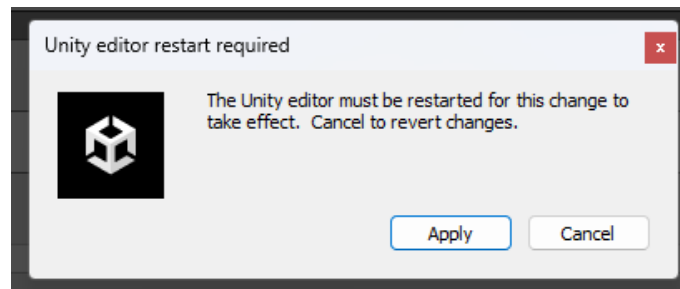
42. Nyalakan *setting* “Run in Background”.



43. Beralih ke *Other Settings*, lalu cari “Active Input Handling” kemudian ganti menjadi *Both*.

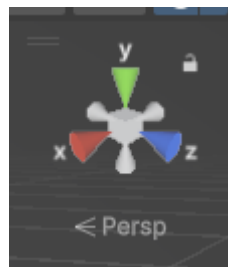


44. *Apply setting*, lalu Unity Editor akan *restart*.

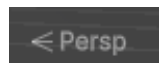


E. Navigasi pada *Unity Editor Scene View*

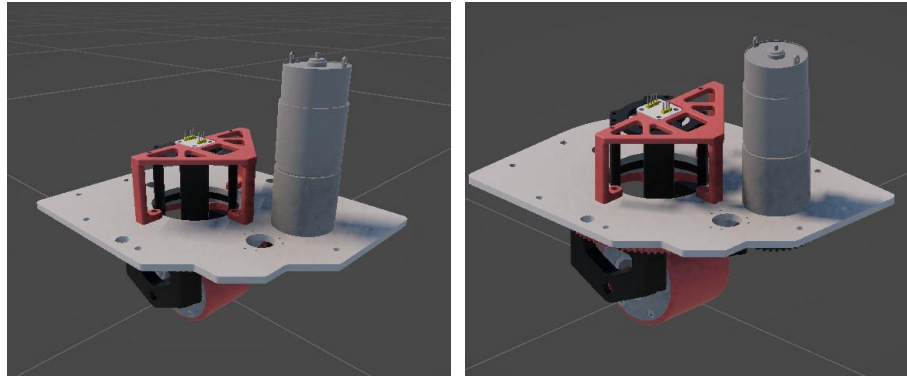
1. Orientasi dunia tiga dimensi pada Unity dapat dilihat pada gizmos yang memiliki tiga sumbu yaitu X, Y, Z. Setiap sumbu memiliki bentuk seperti *cone* yang dapat ditekan untuk memberikan pandangan objek dari tampak misalkan depan, kiri, kanan, dst.



2. Pada gizmos terdapat perspektif tiga dimensi, di Unity terbagi menjadi dua yaitu *Perspective* dan *Ortographic (isometric)*. Untuk beralih antar perspektif dapat dilakukan dengan klik pada Persp atau Iso.



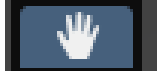
3. Berikut merupakan perbedaan dari mode *perspective* (kiri) dan *ortographic* (kanan).



4. Unity Editor memiliki fitur yang dinamai sebagai *View Tool* seperti pada gambar di bawah ini:



5. Cermati tabel di bawah ini untuk mengetahui fungsi dari tiap fitur di *View Tool*:

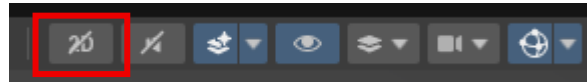
Ikon	Nama Tool	Fungsi	Shortcut
	<i>View Tool</i>	Untuk menggeser sudut pandang kamera di Scene View (pan/rotate/zoom).	Q
	<i>Move Tool</i>	Menggeser (translate) objek	W

		pada sumbu X, Y, atau Z.	
	<i>Rotate Tool</i>	Memutar objek pada sumbu X, Y, atau Z.	E
	<i>Scale Tool</i>	Mengubah ukuran objek secara keseluruhan atau pada sumbu tertentu.	R
	<i>Rect Tool</i>	Digunakan untuk UI; mengatur posisi dan ukuran RectTransform.	T
	<i>Transform Tool</i>	Kombinasi Move, Rotate, dan Scale (opsional, tergantung Unity version).	Y

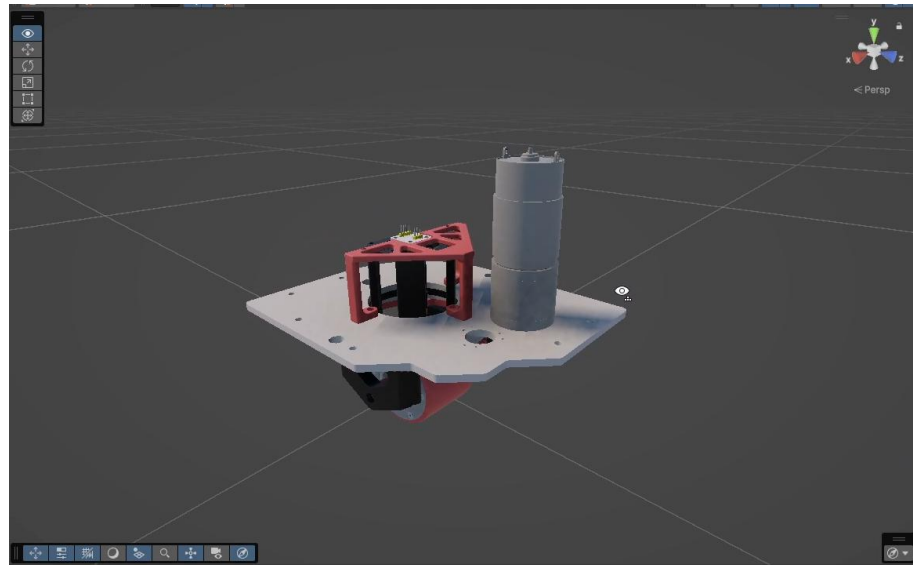
Tabel 1. Fungsi Tombol *View Tools* di Unity

- Selain menggunakan View Tools, Unity juga memiliki mode *Flythrough* untuk bernavigasi pada *Scene View*. Pastikan tampilan *Scene* berada dalam *Perspective* untuk memulai mode ini.

7. Klik tombol 3D/2D di atas kanan *Scene View* jika diperlukan untuk berganti ke 3D.

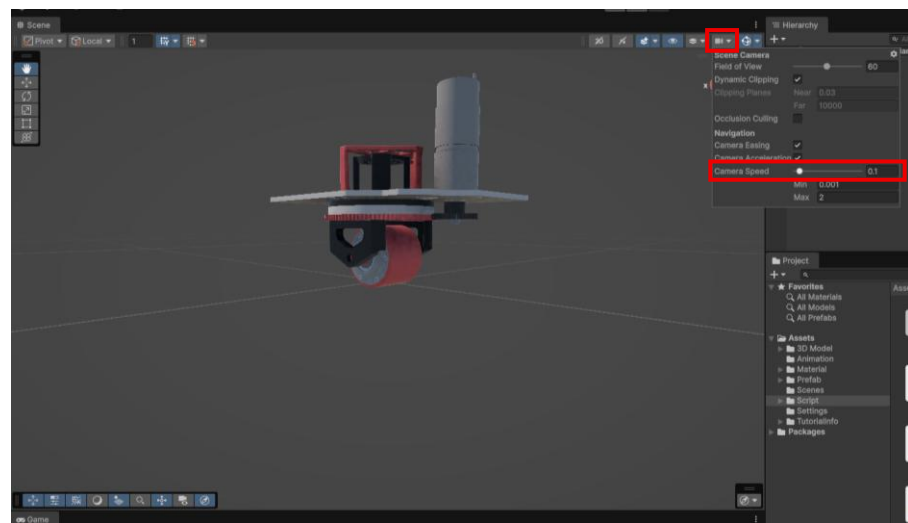


8. Untuk masuk ke *Flythrough Mode* klik dan tahan tombol kanan *mouse* di *Scene View* hingga cursor berubah menjadi *icon* mata seperti pada gambar di bawah ini.

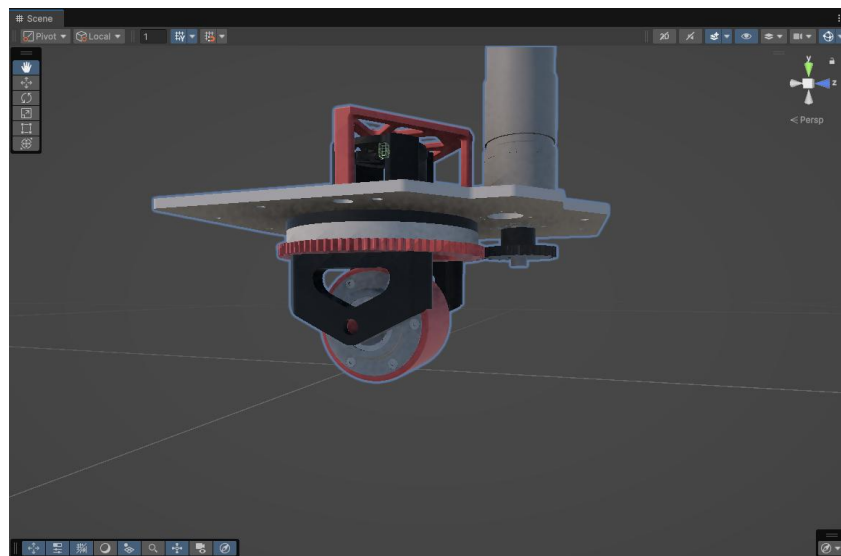


9. Untuk melakukan navigasi dalam *Flythrough Mode*, sambil menahan tombol kanan *mouse*, gunakan kombinasi berikut:
- Tekan W untuk maju
 - Tekan S untuk mundur
 - Tekan A untuk ke kiri
 - Tekan D untuk ke kanan
 - Tekan Q untuk turun
 - Tekan E untuk naik

10. Gerakan maju, mundur, dan sebagainya dapat dipercepat dengan tekan dan tahan tombol *Shift* bersamaan dengan tombol arah untuk mempercepat pergerakan kamera.
11. Jika gerakan kamera terlalu cepat, ubah *Camera Speed* dengan menekan *icon* kamera di atas kanan window *Scene*, lalu geser *slider* ke kiri untuk memperlambat atau kanan untuk mempercepat.

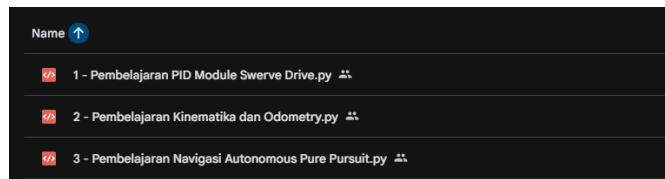


12. Gunakan mode *Flythrough* untuk mendapatkan pandangan seperti pada gambar di bawah ini:

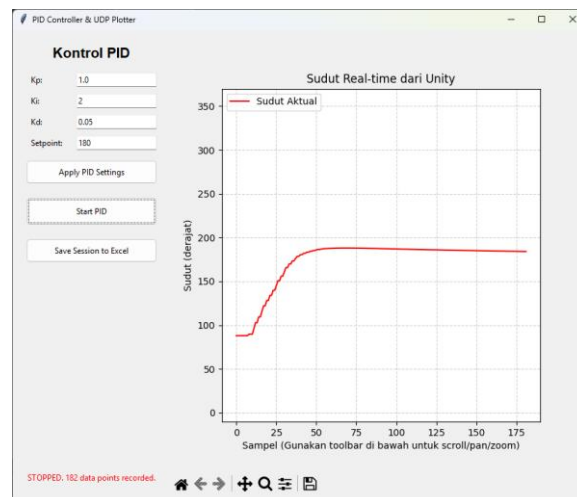


F. Pemrograman GUI dan *Data Acquisition* Python

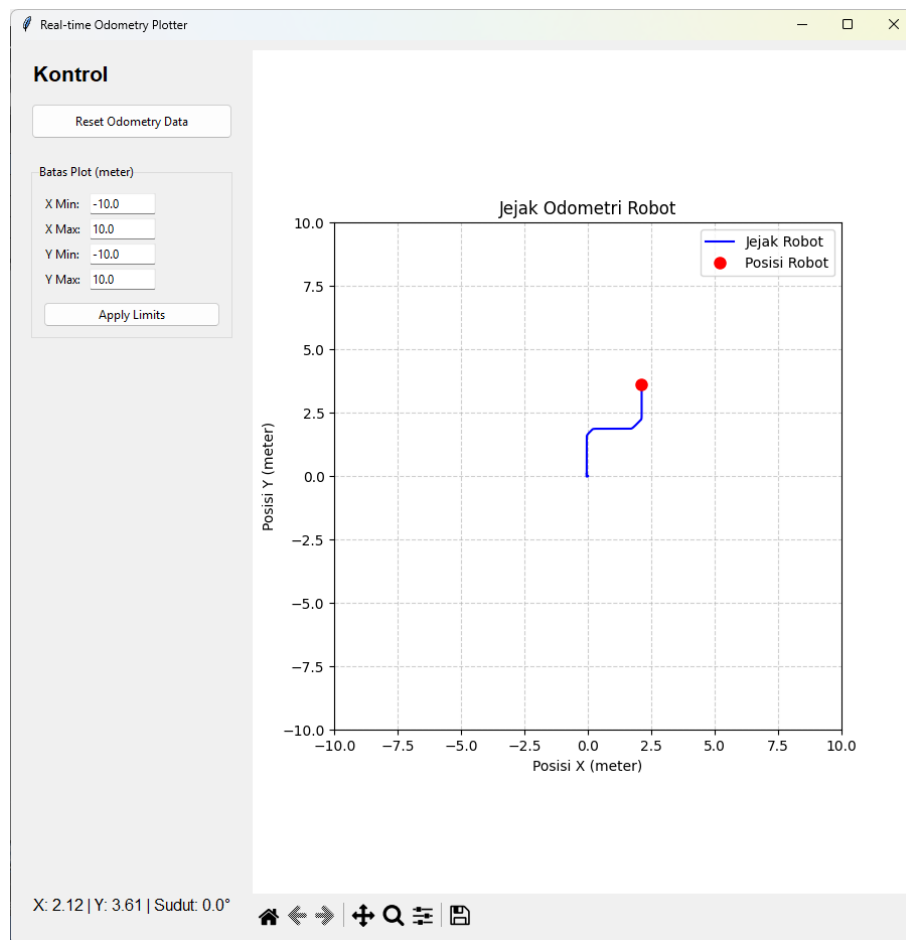
Pemrograman Python terbagi menjadi tiga *file* yang dapat diunduh pada laman <https://bit.ly/3IGCY0b>. Setiap *file* akan digunakan untuk tiap pertemuan pada pembelajaran Simulator Kinematika dan Navigasi Swerve Drive AMR.



Praktikum simulasi robotika akan dilaksanakan dalam tiga pertemuan secara berjenjang mulai dari Pembelajaran Module *Swerve Drive* yang akan membahas tentang respon sistem dan *tuning* PID pada motor gerak rotasi *Swerve Drive*, program Python dapat digunakan untuk berkomunikasi dengan Unity untuk tuning parameter PID, melihat respon sistem melalui grafik *realtime*, serta menyimpan data untuk analisis lebih lanjut terkait respon sistem. Berikut merupakan gambar yang menunjukkan tampilan GUI untuk program Python pembelajaran pertama.

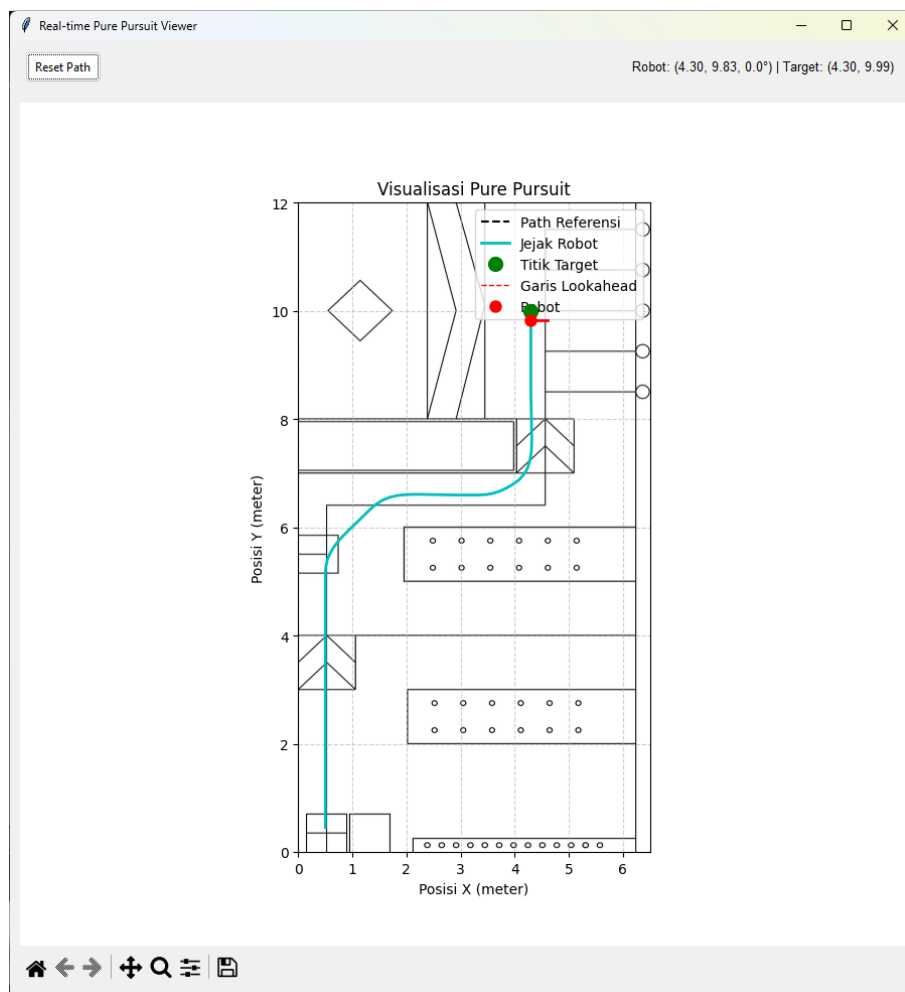


Pembelajaran dua akan membahas tentang pengaplikasian dari persamaan kinematika untuk mengendalikan pergerakan robot seperti maju, mundur, menyamping, dan berotasi. Robot dapat digerakkan menggunakan tombol *keyboard* atau menggunakan *joystick*, kemudian pergerakan robot dalam lapangan dapat diamati melalui telemetri program Python yang sekaligus mengkonfirmasi kebenaran dari persamaan kinematika yang diaplikasikan. Berikut merupakan gambar dari program Python kedua.



Pembelajaran ketiga membahas tentang navigasi *autonomous* menggunakan algoritma *pure pursuit*. Robot akan bergerak secara otonom melalui jalur yang telah disediakan, program *Python* dapat memantau

pergerakan robot melalui jalur yang telah disediakan, jalur didapatkan melalui algoritma *Pathfinding A**. Tuning parameter *Pure Pursuit* dapat dilakukan di Unity, sehingga program *Python* dapat merekam lintasan yang dibuat robot untuk bergerak secara otomatis ke lantai 3 pada lapangan berdasarkan kompetisi ABU Robocon 2024. Melalui data pengamatan yang disediakan oleh program *Python* dapat dilakukan *tuning* untuk parameter *Pure Pursuit*. Berikut merupakan gambar dari program *Python* pada pembelajaran ketiga.



G. Kode Program Python

1. Kode Python Pembelajaran PID Module Swerve Drive

```
import socket
import threading
import queue
import time
import tkinter as tk
from tkinter import ttk, filedialog, messagebox #
Ditambahkan messagebox untuk notifikasi error
from collections import deque
import pandas as pd
import matplotlib.animation as animation # <--- PERBAIKAN DI
SINI
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import
FigureCanvasTkAgg, NavigationToolbar2Tk

# --- Konfigurasi dipindahkan ke dalam kelas atau sebagai
konstanta ---
PYTHON_IP = '127.0.0.1'
PYTHON_PORT_LISTEN = 5005
UNITY_IP = '127.0.0.1'
UNITY_PORT_SEND = 5006
MAX_DATA_POINTS_PLOT = 200

# --- Listener Function (Lebih baik sebagai metode statis
atau di luar kelas) ---
# Tidak ada perubahan besar di sini, sudah cukup baik.
def udp_listener_thread(sock, q, stop_event):
    """Fungsi ini berjalan di thread terpisah untuk
mendengarkan data UDP."""
    print("UDP listener thread dimulai.")
    while not stop_event.is_set():
        try:
            sock.settimeout(1.0)
            data_bytes, _ = sock.recvfrom(1024)
            data_str = data_bytes.decode('utf-8')
            q.put((time.time(), float(data_str)))
        except socket.timeout:
            continue
        except (ValueError, UnicodeDecodeError):
            print("Peringatan: Menerima data UDP tidak
valid.")
```

```

        continue
    except Exception as e:
        if not stop_event.is_set():
            print(f"Error kritis di UDP listener: {e}")
        break
    print("UDP listener thread dihentikan.")

class PIDControllerApp:
    def __init__(self, master):
        self.master = master
        self.master.title("PID Controller & UDP Plotter")
        self.master.geometry("850x700")

        # --- Inisialisasi Atribut Kelas (Enkapsulasi) ---
        self.sock = self._setup_socket()
        self.data_queue = queue.Queue()
        self.stop_event = threading.Event()

        # Data untuk plot dan rekaman
        self.plot_data = deque(maxlen=MAX_DATA_POINTS_PLOT)
        self.session_recorder = []
        self.session_pid_params = {}

        # State aplikasi
        self.is_running = False
        self.start_time = 0
        self.current_setpoint = 180.0

        # Panggil metode untuk membangun UI dan memulai
listener
        self._create_widgets()
        self._start_listener_thread()
        self.master.protocol("WM_DELETE_WINDOW",
self._on_closing)

    def _setup_socket(self):
        """Membungkus setup socket dalam satu metode."""
        try:
            sock = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
            sock.bind((PYTHON_IP, PYTHON_PORT_LISTEN))
            return sock
        except OSError as e:

```

```

        messagebox.showerror("Socket Error", f"Gagal
bind ke port {PYTHON_PORT_LISTEN}.\nPastikan tidak ada
aplikasi lain yang menggunakan port ini.\n\nError: {e}")
        self.master.destroy()
        raise

    def _create_widgets(self):
        """Membangun semua elemen GUI dalam satu metode
terpusat."""
        # --- Frame Kontrol (Kiri) ---
        control_frame = ttk.Frame(self.master, padding="10")
        control_frame.pack(side=tk.LEFT, fill=tk.Y, padx=10,
pady=10)

        ttk.Label(control_frame, text="Kontrol PID",
font=("Helvetica", 16, "bold")).pack(pady=(0, 10))

        # Variabel Tkinter
        self.kp_var = tk.StringVar(value="1.0")
        self.ki_var = tk.StringVar(value="0.1")
        self.kd_var = tk.StringVar(value="0.05")
        self.setpoint_var =
tk.StringVar(value=str(self.current_setpoint))

        # Membuat input fields menggunakan metode helper
        self._create_entry(control_frame, "Kp:",
self.kp_var)
        self._create_entry(control_frame, "Ki:",
self.ki_var)
        self._create_entry(control_frame, "Kd:",
self.kd_var)
        self._create_entry(control_frame, "Setpoint:",
self.setpoint_var)

        ttk.Button(control_frame, text="Apply PID Settings",
command=self.apply_pid_settings).pack(pady=10, ipady=5,
fill=tk.X)

        self.start_stop_button = ttk.Button(control_frame,
text="Start PID", command=self.toggle_pid_control)
        self.start_stop_button.pack(pady=10, ipady=8,
fill=tk.X)

```

```

        self.save_button = ttk.Button(control_frame,
text="Save Session to Excel", command=self.save_to_excel,
state=tk.DISABLED)
        self.save_button.pack(pady=10, ipady=5, fill=tk.X)

        self.status_label = ttk.Label(control_frame,
text="Status: Ready", foreground="blue", wraplength=200) #
wraplength untuk pesan error panjang
        self.status_label.pack(side=tk.BOTTOM, pady=10)

        # --- Frame Plot (Kanan) ---
        plot_frame = ttk.Frame(self.master)
        plot_frame.pack(side=tk.RIGHT, fill=tk.BOTH,
expand=True, padx=(0,10), pady=10)

        self.fig, self.ax =
plt.subplots(facecolor='#f0f0f0') # Samakan warna background
        self.ax.set_facecolor('#ffffff')

        self.canvas = FigureCanvasTkAgg(self.fig,
master=plot_frame)

        toolbar = NavigationToolbar2Tk(self.canvas,
plot_frame, pack_toolbar=False)
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        self.canvas.get_tk_widget().pack(side=tk.TOP,
fill=tk.BOTH, expand=True)

        # Simpan referensi animasi untuk mencegah garbage
collection
        self._animation = animation.FuncAnimation(self.fig,
self._update_plot, interval=50, blit=False)

        def _create_entry(self, parent, label_text,
text_variable):
            """Metode helper untuk membuat label dan entry."""
            frame = ttk.Frame(parent)
            frame.pack(fill=tk.X, pady=5)
            ttk.Label(frame, text=label_text,
width=10).pack(side=tk.LEFT, padx=5)
            ttk.Entry(frame, textvariable=text_variable,
width=15).pack(side=tk.LEFT, expand=True, fill=tk.X)

        def _start_listener_thread(self):

```

```

        self.listener_thread =
threading.Thread(target=udp_listener_thread,
args=(self.sock, self.data_queue, self.stop_event))
        self.listener_thread.daemon = True
        self.listener_thread.start()

# --- Logika Aplikasi ---
def toggle_pid_control(self):
    if self.is_running:
        # Proses STOP
        self._send_command_to_unity("stop_pid")
        self.status_label.config(text=f"STOPPED.
{len(self.session_recorder)} data points recorded.",
foreground="red")
        self.start_stop_button.config(text="Start PID")
        self.save_button.config(state=tk.NORMAL)
        self.is_running = False
    else:
        # Proses START
        if not
self.apply_pid_settings(is_starting=True):
            # Jangan mulai jika parameter awal tidak
valid
            return

        self._reset_session_data()
        self._send_command_to_unity("start_pid")
        self.status_label.config(text="Status: PID
Running", foreground="green")
        self.start_stop_button.config(text="Stop PID")
        self.save_button.config(state=tk.DISABLED)
        self.start_time = time.time()
        self.is_running = True

def apply_pid_settings(self, is_starting=False):
    """Mengirim pengaturan PID ke Unity. Mengembalikan
True jika sukses."""
    try:
        params = {
            'Kp': float(self.kp_var.get()),
            'Ki': float(self.ki_var.get()),
            'Kd': float(self.kd_var.get()),
            'Setpoint': float(self.setpoint_var.get())
        }
    except ValueError:

```



```

        self.status_label.config(text="Error: Input
        harus berupa angka valid.", foreground="red")
        messagebox.showerror("Input Error", "Pastikan
        semua nilai Kp, Ki, Kd, dan Setpoint adalah angka.")
        return False

    if is_starting:
        self.session_pid_params = params

        self.current_setpoint = params['Setpoint']
        message =
        f"pid,{params['Kp']},{params['Ki']},{params['Kd']},{params['
        Setpoint']}"
        self._send_command_to_unity(message)

        if not self.is_running:
            self.status_label.config(text="PID settings
            applied. Ready to start.", foreground="blue")
            return True

    def _send_command_to_unity(self, command):
        try:
            self.sock.sendto(command.encode('utf-8'),
            (UNITY_IP, UNITY_PORT_SEND))
        except Exception as e:
            print(f"Gagal mengirim perintah '{command}':
            {e}")
            self.status_label.config(text=f"Error sending
            command: {e}", foreground="red")

    def _reset_session_data(self):
        self.plot_data.clear()
        self.session_recorder.clear()
        self._update_plot(None)
        print("Grafik dan data rekaman telah direset.")

    def _update_plot(self, frame):
        # 1. Proses antrian data
        if self.is_running:
            while not self.data_queue.empty():
                try:
                    timestamp, value =
                    self.data_queue.get_nowait()
                    self.plot_data.append(value)

```

```

        elapsed_time = timestamp -
self.start_time

        self.session_recorder.append({
            "Waktu (s)": elapsed_time, "Sudut
Aktual": value, "Setpoint": self.current_setpoint
        })
    except queue.Empty:
        break

    # 2. Gambar ulang plot
    self.ax.clear()
    self.ax.grid(True, linestyle='--', alpha=0.6)
    self.ax.set_title("Sudut Real-time dari Unity")
    self.ax.set_xlabel("Sampel (Gunakan toolbar di bawah
untuk scroll/pan/zoom)")
    self.ax.set_ylabel("Sudut (derajat)")
    self.ax.set_ylim(-10, 370)

    if self.plot_data:
        self.ax.plot(range(len(self.plot_data)),
list(self.plot_data), 'r-', label='Sudut Aktual')
        if self.is_running:
            self.ax.axhline(y=self.current_setpoint,
color='b', linestyle='--', label='Setpoint')

    # Selalu tampilkan legenda jika ada label yang
didefinisikan
    if self.ax.get_legend_handles_labels()[0]:
        self.ax.legend(loc='upper left')

    # Perlu dipanggil untuk menggambar ulang canvas
self.canvas.draw_idle()

def save_to_excel(self):
    if not self.session_recorder:
        messagebox.showwarning("No Data", "Tidak ada
data sesi untuk disimpan.")
        return

    filepath =
filedialog.asksaveasfilename(defaultextension=".xlsx",
filetypes=[("Excel Workbook", "*.xlsx")], title="Simpan Sesi
PID")
    if not filepath:

```

```

        self.status_label.config(text="Penyimpanan
dibatalkan.", foreground="orange")
        return

        self.status_label.config(text="Menyimpan ke
Excel...", foreground="blue")
        self.master.update_idletasks()

        try:
            df_data = pd.DataFrame(self.session_recorder)
            df_data['Waktu (s)'] = df_data['Waktu
(s)'].round(4)
            df_data['Sudut Aktual'] = df_data['Sudut
Aktual'].round(4)
            df_params =
pd.DataFrame([self.session_pid_params])

            with pd.ExcelWriter(filepath, engine='openpyxl')
as writer:
                df_data.to_excel(writer,
sheet_name='PID_Data', index=False)
                df_params.to_excel(writer,
sheet_name='PID_Parameters', index=False)

            self.status_label.config(text="Data berhasil
disimpan!", foreground="green")
            messagebox.showinfo("Sukses", f"Data berhasil
disimpan ke:\n{filepath}")
        except Exception as e:
            self.status_label.config(text=f"Error saat
menyimpan: {e}", foreground="red")
            messagebox.showerror("Save Error", f"Gagal
menyimpan file Excel.\n\nError: {e}")

    def _on_closing(self):
        """Menangani penutupan jendela aplikasi dengan
bersih."""
        if messagebox.askokcancel("Keluar", "Apakah Anda
yakin ingin keluar?"):
            print("Menutup aplikasi...")
            self.stop_event.set()
            if self.sock:
                self.sock.close()
            # Beri waktu sedikit untuk thread selesai
            if hasattr(self, 'listener_thread'):

```

```

        self.listener_thread.join(timeout=1.5)
        self.master.destroy()

if __name__ == "__main__":
    try:
        root = tk.Tk()
        app = PIDControllerApp(root)
        root.mainloop()
    except Exception as e:
        print(f"Gagal menjalankan aplikasi: {e}")

```

2. Kode Python Pembelajaran Kinematika dan Odometry

```

import socket
import threading
import queue
import tkinter as tk
from tkinter import ttk, messagebox
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.backends.backend_tkagg import
FigureCanvasTkAgg, NavigationToolbar2Tk

# --- Konfigurasi UDP ---
UDP_IP = '127.0.0.1'
UDP_PORT = 5005

# --- Fungsi Listener UDP (Tidak ada perubahan) ---
def udp_listener_thread(q, sock, stop_event):
    """Mendengarkan data UDP dan memasukkannya ke dalam
    antrian."""
    print("UDP listener thread dimulai.")
    while not stop_event.is_set():
        try:
            sock.settimeout(1.0)
            data_bytes, _ = sock.recvfrom(1024)
            data_str = data_bytes.decode('utf-8')
            parts = data_str.split(',')
            if len(parts) == 3:
                x, y, angle = map(float, parts)
                q.put((x, y, angle))
        except socket.timeout:
            continue
        except (ValueError, IndexError):
            print(f"Menerima data tidak valid: {data_str}")
        except Exception as e:

```

```

        if not stop_event.is_set():
            print(f"Error di listener UDP: {e}")
            break
    print("UDP listener thread dihentikan.")

class OdometryPlotterApp:
    def __init__(self, master):
        self.master = master
        master.title("Real-time Odometry Plotter")
        master.geometry("900x900") # Sedikit lebih lebar
        untuk input baru

        # --- Inisialisasi data dan state ---
        self.x_path = []
        self.y_path = []
        self.current_x, self.current_y, self.current_angle =
0.0, 0.0, 0.0

        # --- Setup Networking ---
        self.data_queue = queue.Queue()
        self.stop_event = threading.Event()
        self.sock = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        self.sock.bind((UDP_IP, UDP_PORT))

        # <<< TAMBAHAN >>>: Variabel untuk batas plot statis
        self.x_min_var = tk.StringVar(value="-10.0")
        self.x_max_var = tk.StringVar(value="10.0")
        self.y_min_var = tk.StringVar(value="-10.0")
        self.y_max_var = tk.StringVar(value="10.0")

        self._create_widgets()
        self._start_listener()

        # Terapkan batas awal saat aplikasi dimulai
        self._apply_plot_limits()

        self._animation = animation.FuncAnimation(self.fig,
self._update_plot, interval=100, blit=False)
        self.master.protocol("WM_DELETE_WINDOW",
self._on_closing)

    def _create_widgets(self):
        """Membuat semua elemen GUI."""
        # --- Frame Kontrol (Kiri) ---

```

```

        control_frame = ttk.Frame(self.master, padding="10")
        control_frame.pack(side=tk.LEFT, fill=tk.Y, padx=10,
pady=10)

        ttk.Label(control_frame, text="Kontrol",
font=("Helvetica", 16, "bold")).pack(pady=(0, 10),
anchor='w')
        ttk.Button(control_frame, text="Reset Odometry
Data", command=self._reset_data).pack(fill=tk.X, pady=5,
ipady=5)

        # <<< TAMBAHAN >>>: Frame dan input untuk batas plot
        limits_frame = ttk.LabelFrame(control_frame,
text="Batas Plot (meter)", padding="10")
        limits_frame.pack(fill=tk.X, pady=20)

        self._create_limit_entry(limits_frame, "X Min:",
self.x_min_var)
        self._create_limit_entry(limits_frame, "X Max:",
self.x_max_var)
        self._create_limit_entry(limits_frame, "Y Min:",
self.y_min_var)
        self._create_limit_entry(limits_frame, "Y Max:",
self.y_max_var)

        ttk.Button(limits_frame, text="Apply Limits",
command=self._apply_plot_limits).pack(fill=tk.X,
pady=(10,0))

        self.status_label = ttk.Label(control_frame,
text="Menunggu data...", font=("Helvetica", 12),
wraplength=200)
        self.status_label.pack(side=tk.BOTTOM, pady=10)

        # --- Frame Plot (Kanan) ---
        plot_frame = ttk.Frame(self.master)
        plot_frame.pack(side=tk.RIGHT, fill=tk.BOTH,
expand=True, padx=(0, 10), pady=10)

        self.fig, self.ax = plt.subplots()
        self.ax.set_aspect('equal', adjustable='box')
        self.ax.set_title("Jejak Odometri Robot")
        self.ax.set_xlabel("Posisi X (meter)")
        self.ax.set_ylabel("Posisi Y (meter)")
        self.ax.grid(True, linestyle='--', alpha=0.6)

```

```

        self.path_line, = self.ax.plot([], [], 'b-',
label="Jejak Robot")
        self.robot_marker, = self.ax.plot([], [], 'ro',
markersize=8, label="Posisi Robot")
        self.ax.legend()

        self.canvas = FigureCanvasTkAgg(self.fig,
master=plot_frame)
        toolbar = NavigationToolbar2Tk(self.canvas,
plot_frame, pack_toolbar=False)
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH,
expand=True)

    def _create_limit_entry(self, parent, label_text,
text_variable):
        """Helper untuk membuat entry batas."""
        frame = ttk.Frame(parent)
        frame.pack(fill=tk.X, pady=2)
        ttk.Label(frame, text=label_text,
width=7).pack(side=tk.LEFT)
        ttk.Entry(frame, textvariable=text_variable,
width=10).pack(side=tk.LEFT)

    def _start_listener(self):
        """Memulai thread listener UDP."""
        self.listener_thread = threading.Thread(
            target=udp_listener_thread,
            args=(self.data_queue, self.sock,
self.stop_event)
        )
        self.listener_thread.daemon = True
        self.listener_thread.start()

    def _update_plot(self, frame):
        """Fungsi yang dipanggil secara berkala untuk
memperbarui plot."""
        data_updated = False
        while not self.data_queue.empty():
            try:
                x, y, angle = self.data_queue.get_nowait()
                self.x_path.append(x)
                self.y_path.append(y)

```

```

        self.current_x, self.current_y,
self.current_angle = x, y, angle
        data_updated = True
    except queue.Empty:
        break

    if data_updated:
        status_text = f"X: {self.current_x:.2f} | Y:
{self.current_y:.2f} | Sudut: {self.current_angle:.1f}°"
        self.status_label.config(text=status_text)

        # Update data di plot
        self.path_line.set_data(self.x_path,
self.y_path)
        self.robot_marker.set_data([self.current_x],
[self.current_y])

        # <<< PERBAIKAN >>>: Hapus rescaling otomatis
        # self.ax.relim()
        # self.ax.autoscale_view()

        # Gambar ulang canvas
        self.canvas.draw_idle()

def _apply_plot_limits(self):
    """Menerapkan batas plot dari input GUI."""
    try:
        x_min = float(self.x_min_var.get())
        x_max = float(self.x_max_var.get())
        y_min = float(self.y_min_var.get())
        y_max = float(self.y_max_var.get())

        if x_min >= x_max or y_min >= y_max:
            messagebox.showerror("Input Error", "Nilai
Min harus lebih kecil dari nilai Max.")
            return

        self.ax.set_xlim(x_min, x_max)
        self.ax.set_ylim(y_min, y_max)

        # Gambar ulang canvas untuk menerapkan batas
baru
        self.canvas.draw_idle()
        print(f"Batas plot diatur ke: X({x_min},
{x_max}), Y({y_min}, {y_max})")

```



```

        except ValueError:
            messagebox.showerror("Input Error", "Batas plot
            harus berupa angka yang valid.")

    def _reset_data(self):
        """Menghapus semua data odometri dan membersihkan
        plot."""
        print("Mereset data odometri...")
        self.x_path.clear()
        self.y_path.clear()

        self.path_line.set_data([], [])
        self.robot_marker.set_data([], [])

        # <<< PERBAIKAN >>>: Tidak perlu rescale, cukup
        gambar ulang
        self.canvas.draw_idle()

        self.status_label.config(text="Data direset.
        Menunggu data baru...")

    def _on_closing(self):
        """Menangani penutupan aplikasi dengan bersih."""
        print("Menutup aplikasi...")
        self.stop_event.set()
        self.sock.close()
        if hasattr(self, 'listener_thread') and
        self.listener_thread.is_alive():
            self.listener_thread.join(timeout=1.5)
            self.master.destroy()

if __name__ == "__main__":
    root = tk.Tk()
    app = OdometryPlotterApp(root)
    root.mainloop()

```

3. Kode Python Pembelajaran Navigasi Autonomous Pure Pursuit

```

import socket
import threading
import queue
import tkinter as tk
from tkinter import ttk, messagebox
import matplotlib.pyplot as plt
import matplotlib.animation as animation

```

```

from matplotlib.backends.backend_tkagg import
FigureCanvasTkAgg, NavigationToolbar2Tk
from PIL import Image, UnidentifiedImageError
from math import cos as _cos, sin as _sin, radians as _rad
import pandas as pd

# --- KONFIGURASI APLIKASI ---
UDP_IP = '127.0.0.1'
UDP_PORT = 5005
TRACK_IMAGE_PATH = r"D:\Data
Kuliah\maestro2024\PathPlanning\lapangan_gui.png"
REFERENCE_PATH_FILE = "path.csv"
TARGET_WIDTH = 12000
TARGET_HEIGHT = 6500

# --- FUNGSI LISTENER UDP ---
def udp_listener_thread(q, sock, stop_event):
    print("UDP listener thread dimulai.")
    while not stop_event.is_set():
        try:
            sock.settimeout(1.0)
            data_bytes, _ = sock.recvfrom(1024)
            data_str = data_bytes.decode('utf-8')
            parts = data_str.split(',')
            if len(parts) == 5:
                values = tuple(map(float, parts))
                q.put(values)
        except socket.timeout:
            continue
        except (ValueError, IndexError):
            print(f"Peringatan: Menerima data UDP tidak
valid: {data_str}")
        except Exception as e:
            if not stop_event.is_set():
                print(f"Error kritis di listener UDP: {e}")
            break
    print("UDP listener thread dihentikan.")

# --- KELAS UTAMA APLIKASI GUI ---
class PurePursuitViewerApp:
    def __init__(self, master):
        self.master = master
        master.title("Real-time Pure Pursuit Viewer")
        master.geometry("900x950")

```

```

        self.robot_path_x, self.robot_path_y = [], []

        self.data_queue = queue.Queue()
        self.stop_event = threading.Event()
        self.sock = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        try:
            self.sock.bind((UDP_IP, UDP_PORT))
        except OSError as e:
            messagebox.showerror("Socket Error", f"Gagal
bind ke port {UDP_PORT}.\n\nError: {e}")
            self.master.destroy()
            raise

        self._create_widgets()
        self._load_and_process_track_image(TRACK_IMAGE_PATH)
        self._load_reference_path(REFERENCE_PATH_FILE)
        self._start_listener()

        self._animation = animation.FuncAnimation(self.fig,
self._update_plot, interval=50, blit=True,
init_func=self._init_plot)
        self.master.protocol("WM_DELETE_WINDOW",
self._on_closing)

    def _create_widgets(self):
        control_frame = ttk.Frame(self.master, padding="10")
        control_frame.pack(side=tk.TOP, fill=tk.X)

        ttk.Button(control_frame, text="Reset Path",
command=self._reset_data).pack(side=tk.LEFT, padx=5,
ipady=3)

        self.status_label = ttk.Label(control_frame,
text="Menunggu data dari Unity...", font=("Helvetica", 10),
wraplength=500)
        self.status_label.pack(side=tk.RIGHT, padx=10)

        plot_frame = ttk.Frame(self.master);
        plot_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
        self.fig, self.ax = plt.subplots();
        self.ax.set_aspect('equal', adjustable='box');
        self.ax.set_title("Visualisasi Pure Pursuit")

```

```

        self.ax.set_xlabel("Posisi X (meter)");
self.ax.set_ylabel("Posisi Y (meter)"); self.ax.grid(True,
linestyle='--', alpha=0.6)

        self.reference_path_line, = self.ax.plot([], [], 'k-
-', linewidth=1.5, label="Path Referensi", zorder=1)

        # <<< PERBAIKAN 1: Tambahkan animated=True untuk
jejak robot >>>
        self.path_line, = self.ax.plot([], [], 'c-',
linewidth=2, label="Jejak Robot", zorder=2, animated=True)

        self.target_marker, = self.ax.plot([], [], 'go',
markersize=10, label="Titik Target", zorder=4,
animated=True)
        self.lookahead_line, = self.ax.plot([], [], 'r--',
linewidth=1, label="Garis Lookahead", zorder=3,
animated=True)
        self.robot_pos_marker, = self.ax.plot([], [], 'ro',
markersize=8, label="Robot", zorder=5, animated=True)
        self.robot_orient_line, = self.ax.plot([], [], 'r-',
linewidth=2, zorder=6, animated=True)

        self.ax.legend(loc='upper right')

        self.canvas = FigureCanvasTkAgg(self.fig,
master=plot_frame)
        toolbar = NavigationToolbar2Tk(self.canvas,
plot_frame, pack_toolbar=False);
toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH,
expand=True)

    def _load_and_process_track_image(self, filepath):
        try: original_img = Image.open(filepath)
        except (FileNotFoundError, UnidentifiedImageError)
as e: messagebox.showerror("Image Load Error", f"Gagal
memuat gambar dari:\n{filepath}\n\nError: {e}"); return
        w, h = original_img.width, original_img.height;
target_size = (TARGET_WIDTH, TARGET_HEIGHT) if w > h else
(TARGET_HEIGHT, TARGET_WIDTH)
        processed_img = original_img.resize(target_size,
Image.Resampling.LANCZOS).transpose(Image.FLIP_LEFT_RIGHT).r
otate(180)

```

```

        img_width_m, img_height_m = processed_img.width /
1000.0, processed_img.height / 1000.0
        plot_extent = [0, img_width_m, 0, img_height_m]
        self.ax.imshow(processed_img, extent=plot_extent,
aspect='equal', zorder=0, origin='lower')
        self.ax.set_xlim(plot_extent[0], plot_extent[1]);
self.ax.set_ylim(plot_extent[2], plot_extent[3]);
self.canvas.draw_idle()

    def _load_reference_path(self, filepath):
        try:
            path_df = pd.read_csv(filepath); ref_x =
path_df['x'].values; ref_y = path_df['y'].values
            self.reference_path_line.set_data(ref_x, ref_y);
self.canvas.draw_idle()
            print(f"Path referensi dimuat dari
'{filepath}'.")
        except FileNotFoundError: print(f"Peringatan: File
path referensi '{filepath}' tidak ditemukan.")
        except Exception as e: messagebox.showerror("Path
File Error", f"Gagal membaca file '{filepath}'.\nPastikan
formatnya benar.\n\nError: {e}")

    def _start_listener(self):
        self.listener_thread =
threading.Thread(target=udp_listener_thread,
args=(self.data_queue, self.sock, self.stop_event))
        self.listener_thread.daemon = True;
self.listener_thread.start()

    def _init_plot(self):
        """Fungsi inisialisasi untuk blitting. Mengembalikan
semua artis yang di-blit."""
        self.robot_pos_marker.set_data([], [])
        self.robot_orient_line.set_data([], [])
        self.target_marker.set_data([], [])
        self.lookahead_line.set_data([], [])
        self.path_line.set_data([], []) # <<< PERBAIKAN 2:
Inisialisasi jejak di sini juga

        # Kembalikan semua elemen yang memiliki
animated=True
        return self.path_line, self.robot_pos_marker,
self.robot_orient_line, self.target_marker,
self.lookahead_line

```

```

def _update_plot(self, frame):
    """Fungsi animasi yang dioptimalkan."""
    latest_data = None
    while not self.data_queue.empty():
        try:
            latest_data = self.data_queue.get_nowait()
        except queue.Empty:
            break

    if latest_data:
        rx, ry, ra, tx, ty = latest_data

        self.robot_path_x.append(rx)
        self.robot_path_y.append(ry)
        self.path_line.set_data(self.robot_path_x,
self.robot_path_y)

        self._update_robot_marker(rx, ry, ra)
        self.target_marker.set_data([tx], [ty])
        self.lookahead_line.set_data([rx, tx], [ry, ty])

        status_text = f"Robot: ({rx:.2f}, {ry:.2f},
{ra:.1f}°) | Target: ({tx:.2f}, {ty:.2f})"
        self.status_label.config(text=status_text)

        # <<< PERBAIKAN 3: Kembalikan jejak sebagai bagian
dari tuple blit >>>
        return self.path_line, self.robot_pos_marker,
self.robot_orient_line, self.target_marker,
self.lookahead_line

    def _update_robot_marker(self, x, y, angle_deg):
        self.robot_pos_marker.set_data([x], [y]); length =
0.3; angle_rad = _rad(angle_deg)
        end_x = x + length * _cos(angle_rad); end_y = y +
length * _sin(angle_rad)
        self.robot_orient_line.set_data([x, end_x], [y,
end_y])

    def _reset_data(self):
        """Mereset jejak robot yang telah dilalui."""
        if messagebox.askokcancel("Konfirmasi Reset",
"Apakah Anda yakin ingin menghapus semua jejak robot?"):
            print("Mereset jejak robot...")

```

```

        self.robot_path_x.clear()
        self.robot_path_y.clear()
        # Tidak perlu memanggil set_data di sini karena
        _init_plot akan menanganinya
        # saat animasi me-reset.

    def _on_closing(self):
        if messagebox.askokcancel("Keluar", "Apakah Anda
        yakin ingin keluar?"):
            self.stop_event.set(); self.sock.close()
            if hasattr(self, 'listener_thread') and
            self.listener_thread.is_alive():
            self.listener_thread.join(timeout=1.5)
            self.master.destroy()

if __name__ == "__main__":
    try:
        root = tk.Tk()
        app = PurePursuitViewerApp(root)
        root.mainloop()
    except Exception as e:
        import traceback; messagebox.showerror("Fatal
        Error", f"Aplikasi gagal dijalankan.\n\nError: {e}")
        print(f"Gagal menjalankan aplikasi: {e}");
        traceback.print_exc()

```

H. Implementasi Model Kinematika pada Simulasi

1. Implementasi Kinematika Mundur

Pada simulasi terdapat fungsi `SwerveDriveMove` pada skrip `RobotBase.cs`. Fungsi ini menerima perintah gerak global dan menerjemahkannya menjadi perintah sudut dan kecepatan untuk setiap modul roda.

```

public void SwerveDriveMove(double headingAngle, double
dforward, double dstrafe, double drotation)

```

Fungsi ini menerima empat parameter yaitu `headingAngle` untuk sudut orientasi robot saat ini, `dforward` merupakan kecepatan maju yang

diinginkan (V_y), $dstrafe$ merupakan kecepatan menyamping yang diinginkan (V_x), dan $drotation$ adalah kecepatan rotasi yang diinginkan (ω).

```
// Transformasi kecepatan ke kerangka robot berdasarkan heading
double temp = dforward * Math.Cos(Mathf.Deg2Rad * headingAngle) + dstrafe * Math.Sin(Mathf.Deg2Rad * headingAngle);

double strafe = -dforward * Math.Sin(Mathf.Deg2Rad * headingAngle) + dstrafe * Math.Cos(Mathf.Deg2Rad * headingAngle);

double forward = temp;
strafe = -strafe;
```

Pada cuplikan kode di atas digunakan untuk transformasi *Robot-Centric* ke *Field-Centric*, bagian ini melakukan transformasi koordinat. Perintah gerak seperti $dforward$ dan $dstrafe$ biasanya diberikan dalam kerangka acuan robot (*robot-centric*). Kode tersebut mengubahnya menjadi kerangka acuan lapangan (*field-centric*) dengan memperhitungkan $headingAngle$ robot saat ini. Memastikan bahwa “maju” selalu berarti maju ke arah depan lapangan.

```
// Panjang dan lebar robot (dalam meter)
double L = 0.65;
double W = 0.65;

double R = Math.Sqrt(L * L + W * W);

// Perhitungan kecepatan vektor masing-masing roda
double A = strafe - rotation * (L / R);
double B = strafe + rotation * (L / R);
double C = forward - rotation * (W / R);
double D = forward + rotation * (W / R);
```


Kode di atas adalah inti dari rumus kinematika mundur A, B, C, dan D adalah komponen vektor kecepatan yang dihitung untuk setiap sudut sasis robot. L dan W adalah setengah dari panjang dan lebar jarak antar roda pada robot, dan R adalah diagonalnya.

```
// Hitung kecepatan roda (magnitudo vektor)
swerveKiriDepan.SetSpeed((float)(Math.Sqrt(B * B + C * C)
* baseSpeedRpm));

float t1 = Mathf.Repeat((float)(Mathf.Rad2Deg *
Math.Atan2(B, C)), 360f); // Kiri Depan
```

Fungsi *SetSpeed* digunakan untuk memberikan kecepatan setiap roda yang dihitung sebagai magnitudo (panjang) dari vektor komponennya menggunakan teorema Pythagoras ($\text{Math.Sqrt}(B*B + C*C)$). Sedangkan, sudut setiap roda dihitung menggunakan fungsi *atan2* dari komponen vektornya, fungsi ini digunakan untuk menghasilkan sudut dalam rentang -180 hingga 180 derajat, yang kemudian dinormalisasi ke 0-360 derajat dengan *Mathf.Repeat*.

2. Implementasi Kinematika Maju

Fungsi *ApplyForwardKinematic* pada skrip *RobotBase.cs* merupakan implementasi dari model kinematika maju. Fungsi ini mengambil keadaan aktual dari setiap roda dan menggunakannya untuk mengestimasi pergerakan robot secara keseluruhan.

```
public void ApplyForwardKinematic()
```

Dilakukan dekomposisi kecepatan roda, hal ini merupakan kebalikan dari kinematika mundur, di sini kecepatan dan sudut aktual

dari setiap roda diambil, kemudian dipecah menjadi komponen pada v_x dan v_y dalam kerangka acuan robot.

```
double v1X = swerveKiriDepan.CalculateWheelSpeed() *  
Math.Cos(Mathf.Deg2Rad * swerveKiriDepan.GetAngle());  
  
double v1Y = swerveKiriDepan.CalculateWheelSpeed() *  
Math.Sin(Mathf.Deg2Rad * swerveKiriDepan.GetAngle());
```

Setelah dilakukan dekomposisi, kecepatan translasi total robot (v_{xTotal} dan v_{yTotal}) diestimasi dengan merata-ratakan komponen kecepatan dari keempat roda. Hasilnya dikalikan dengan *Time.deltaTime* untuk mendapatkan perubahan posisi pada kerangka ini.

```
// Compute translational velocity in mm/s  
double vxTotal = ((v1X + v2X + v3X + v4X) / 4) *  
Time.deltaTime;  
  
double vyTotal = ((v1Y + v2Y + v3Y + v4Y) / 4) *  
Time.deltaTime;
```

Selanjutnya kecepatan rotasi robot dihitung menggunakan rumus kinematika maju yang memperhitungkan kontribusi setiap komponen kecepatan roda terhadap rotasi sasis.

```
const double R = 0.3;  
  
double omega = (v1X + v1Y + v2X - v2Y - v3X - v3Y - v4X +  
v4Y) / (8 * R);
```

Perubahan posisi dan rotasi pada robot yang telah dihitung kemudian diterapkan pada *transform* dari objek robot di Unity, sehingga dapat menggerakkan robot di dalam simulasi. Data posisi inilah yang kemudian dihitung sebagai perpindahan pada odometri.

```
//Menerapkan gerak translasi robot
robotBase.transform.Translate(new Vector3((float)vyTotal,
0f, -(float)vxTotal));

//Menerapkan gerak rotasi robot
robotBase.transform.Rotate(Vector3.up, (float)(omega *
Mathf.Rad2Deg) * Time.deltaTime);
```

DAFTAR PUSTAKA

- Fachrizi, R., Susanto, E., & Mulkan, I. (2024). Mobilisasi Robot Pengantar Makanan Dengan Tiga Roda Omniwheel Dan Odometry, *11*(5), 5460–5463.
- Fahmizal, F., Priyatmoko, A., & Mayub, A. (2022). Implementasi Kinematika Trajectory Lingkaran pada Robot Roda Mecanum. *Jurnal Listrik Instrumentasi dan Elektronika Terapan (JuLIET)*, *3*(1), 25–30. <https://doi.org/10.22146/juliet.v3i1.74760>
- Ilhan Shanjaya, M. I., Endrasmono, J., Ahmad Putra, Z. M., Sutrisno, I., Subiyanto, L., & Munadhif, I. (2024). Pemodelan Matriks Kinematika Pada Robot Three-Wheel Swerve Drive. *Jurnal Elektronika dan Otomasi Industri*, *11*(3), 760–766. <https://doi.org/10.33795/elkolind.v11i3.5500>
- Tselegkaridis, S., & Sapounidis, T. (2021). Simulators in Educational Robotics: A Review. *Education Sciences*, *11*(1), 1–12. <https://doi.org/10.3390/educsci11010011>
- Villaverde, L., & Maneetham, D. (2024). Kinematic and Parametric Modeling of 6DOF(Degree-of-Freedom) Industrial Welding Robot Design and Implementation. *International Journal of Technology*, *15*(4), 1056–1070. <https://doi.org/10.14716/ijtech.v15i4.6559>

